VISION & MISSION OF THE INSTITUTE

VISION

To induce higher planes of learning by imparting technical education with

- International standards
- Applied research
- Creative Ability
- Value based instruction and to emerge as a premiere institute.

MISSION

Achieving academic excellence by providing globally acceptable technical education by forecasting technology through

- Innovative Research and development
- Industry Institute Interaction
- Empowered Manpower

VISION & MISSION OF THE DEPARTMENT

VISION

"To emerge as a center of excellence in education and research."

MISSION

- To establish skill and learning centric infrastructure in thrust areas.
- To develop Robotics and IOT based infrastructure laboratories.
- To organize events through industry institute collaborations and promote innovation.
- To disseminate knowledge through quality teaching learning process.

A Project Report on

"Automated Smart Waste Segregation System with Live Monitoring and Bin Alerts"

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ELECTRONICS & COMMUNICATION ENGINEERING

By

D. SIDDU MAHESWARA RAO	21MH1A04E1
J. SRI SAI DURGA VEERENDRA	21MH1A0425
K. SAI VEERENDRA VASU	21MH1A0430
P. SATYA KRISHNA	21MH1A0453

Under the Esteemed Guidance of

Mr. P. Ramesh, M.Tech, Ph.D

Assistant Professor

Department of Electronics & Communication Engineering

ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY(A)

(*Approved by AICTE, New Delhi & Permanently Affiliated to JNTUK, Kakinada, accredited by NBA & NAAC)*

Recognized by UGC under sections 2(f) & 12(b) of UGC Act, 1956

Aditya Nagar, ADB road, Surampalem, E.G.Dt,A.P-533437

2021-2025

ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY (A)

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that the Project Report entitled

"Automated Smart Waste Segregation System with Live Monitoring and Intelligent Bin Alerts"

being submitted by

D. SIDHU UMA MAHESHWARAO	21MH1A04E1
J. SRI SAI DURGA VEERENDRA	21MH1A0425
K. SAI VEERENDRA VASU	21MH1A0430
P. SATYA KRISHNA	21MH1A0453

In partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Electronics and Communication Engineering, at Aditya College of Engineering is a bonafide work carried out by them under my guidance and supervision. The result embodied in this report has not been submitted to any other University or Institution for the award of any degree or diploma.

Project Guide
Mr. P. Ramesh, M.Tech, Ph.D
Assistant Professor
Department of ECE
Aditya College of Engineering & Technology(A)

Head of the Department
Mrs. Ch. Janaki Devi, M.Tech, (Ph.D)
Associate Professor & HOD
Department of ECE
Aditya College of Engineering & Technology(A)

External Examiner

DECLARATION

We hereby declare that the entire project work embodied in this dissertation entitled "Automated Smart Waste Segregation System with Live Monitoring and Intelligent Bin Alerts" has been independently carried out by us. As per our knowledge, no part of this work has submitted for any degree in any institution, university, and organization previously. We hereby boldly state that to the best of our knowledge our work is free from plagiarism.

Yours sincerely,

D. SIDDU MAHESWARA RAO	21MH1A04E1
J. SRI SAI DURGA VEERENDRA	21MH1A0425
K. SAI VEERENDRA VASU	21MH1A0430
P. SATYA KRISHNA	21MH1A0453

ACKNOWLEDGEMENT

We express our sincere gratitude and heartful thanks to the under stated person for the successful completion of our final project on "Automated Smart Waste Segregation System with Live Monitoring and Bin Alerts"**.**

First and foremost, we wish to thank our beloved guide **Mr. P. Ramesh, M.Tech, Ph.D** for his kind guidance, valuable advices, and utmost care at every stage of our final project.

We would like to thank Associate Professor **Mrs. Ch. Janaki Devi, M.Tech, (Ph.D)** Head of the department of E.C.E who have provided vital information, which was necessary for success of the project.

The credit in successful completion of our final project goes to highly esteemed Professor **Dr. A. RAMESH, M.Tech, Ph.D Principal of ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY (A)** for his kind guidance, commendable and noteworthy advices, and even memorable encouragement to the same.

Yours sincerely,

D. SIDDU MAHESWARA RAO 21MH1A04E1
J. SRI SAI DURGA VEERENDRA 21MH1A0425
K. SAI VEERENDRA VASU 21MH1A0430
P. SATYA KRISHNA 21MH1A0453

ABSTRACT

Our project “Automated Smart Waste Segregation System with Live Monitoring and Bin Alerts” aims to revolutionize waste management through automation and real-time monitoring. Our system uses a conveyor belt to transport waste to various sensors—moisture, inductive and ultrasonic which classify it into biodegradable, metallic, or plastic. Based on detection, a NEMA 17 stepper motor rotates the platform to direct waste into the appropriate bin.

Each bin is equipped with an ultrasonic sensor to measure the fill level. Once full, a notification is sent to the mobile app via Firebase Cloud Messaging through NodeMCU (ESP8266). The system also features a buzzer alert and an emergency stop button, controllable both physically and remotely via the mobile app. An I2C LCD displays the real-time waste type.

This project combines embedded systems, IoT, and automation for an efficient, eco-friendly solution to waste segregation with minimal human intervention.

INDEX

CONTENTS	PAGE NO
CHAPTER 1: INTRODUCTION	**1-4**
1.1 Motivation	1
1.2 Objective	2
1.3 Existing System	2
1.4 Proposed System	3
1.5 Organization of project	4
CHAPTER 2: LITERATURE SURVEY	**5-6**
CHAPTER 3: HARDWARE COMPONENTS	**7-30**
3.1 Arduino Mega	7
3.2 YL-69 Moisture sensor	12
3.3 Metal sensor	14
3.4 HC SR04 Ultrasonic sensor	15
3.5 Conveyor belt	16
3.6 Buzzer	17
3.7 DC Motor	18
3.8 I2C LCD	19
3.9 L298N DC motor driver	20
3.10 Ball Bearings	21
3.11 NEMA 17 Stepper motor	22
3.12 A4988 Stepper motor driver	24
3.13 MG995 Servo motor	25
3.14 ESP8266	27
3.15 12V 5A SMPS (Switched Mode Power Supply)	30

CHAPTER 4: SOFTWARE REQUIREMENTS **31-35**

4.1 Arduino Software 30

4.1.1 Required Specifications 30

4.2 Program Execution 31

4.2.1 Trouble Shooting 31

CHAPTER 5: DESIGN & IMPLEMENTATION **36-44**

5.1 Block Diagram of Automated Smart Waste Segregation System with Live Monitoring and Bin Alerts 36

5.2 Interfacing buzzer with Arduino Mega 37

5.3 Interfacing Sensors with Arduino Mega 38

5.4 Interfacing DC Motor and L298N Driver Module with Arduino Mega 38

5.5 Interfacing NEMA 17 Stepper Motor with A4988 Driver and Arduino Mega 40

5.6 Interfacing I2C LCD Display with Arduino Mega 41

5.7 Interfacing Ultrasonic Sensors with ESP8266 for Bin Level Monitoring 42

5.8 Integration of ESP8266 NodeMCU and Arduino Mega 43

CHAPTER 6: RESULTS & VALIDATIONS **45-46**

6.1 Prototype of the Automated Smart Waste Segregation System with Live Monitoring and Bin Alerts 45

6.2 Waste Detection and Segregation Output 46

6.3 Bin Level Monitoring Output via Ultrasonic Sensors 46

CHAPTER 7: CONCLUSION & FUTURE SCOPE **47-48**

7.1 Conclusion 47

7.2 Future Scope 47

REFERENCES 49-50

APPENDIX CODE 51-57

LIST OF FIGURES

FIG.NO.	TITLE	PAGE NO
3.1	Arduino Mega Board	7
3.2	Pinout Diagram of Arduino Mega	9
3.3	Pin Diagram of Arduino Mega	10
3.4	Dimensions diagram of Arduino Mega	12
3.5	Moisture sensor	13
3.6	Metal sensor	14
3.7	HC-SR04 Ultrasonic Sensor	15
3.8	Conveyor Belt	16
3.9	Buzzer	17
3.10	DC Motor	18
3.11	I2C LCD Display	19
3.12	L298N Motor Driver Module	20
3.13	Ball Bearings	22
3.14	NEMA 17 Stepper motor	23
3.15	A4988 Stepper motor driver	24
3.16	Servo motor	25
3.17	ESP8266	27
3.18	Pinout of ESP8266	28
3.19	12V 5A SMPS (Switched Mode Power Supply)	30
5.1	Block diagram	36
5.2	Interfacing buzzer with Arduino Mega	37
5.4	Interfacing DC Motor + L298N with Arduino Mega	39
5.5	Interfacing NEMA 17 + A4988 with Arduino Mega	40
5.6	Interfacing I2C LCD Display with Arduino Mega	41
5.7	Interfacing Ultrasonic Sensors with ESP8266	42
5.8	Integration of ESP8266 NodeMCU and Arduino Mega	43
6.1	Prototype of Automated Smart Waste Segregation With live monitoring and bin alerts	45
6.2	Waste Detection and Segregation Output	46
6.3	Bin Level Monitoring Output via Ultrasonic Sensors	46

CHAPTER 1

INTRODUCTION

1.1 MOTIVATION

In today's world, waste management has become a serious concern due to the large amount of garbage produced every day. With increasing population and rapid urban development, the amount of waste being generated is growing faster than ever before. Unfortunately, most of this waste ends up mixed together, making it harder to recycle and more harmful to the environment. Improper segregation not only pollutes land and water but also puts a huge burden on landfill sites and waste workers.

We observed that in many places, waste is still being sorted by hand, which is not only slow and inefficient but also exposes people to health risks. Manual segregation often leads to errors and delays in processing, especially when recyclable and non-recyclable materials are mixed. This makes the entire waste management process costly, unsafe, and harmful to the environment.

These issues inspired us to come up with a solution that can reduce human involvement and make the process faster and smarter. Our Automated Smart Waste Segregation System with Live Monitoring and Intelligent Bin Alerts aims to automate the identification and separation of different types of waste using sensors and basic electronics. We believe that by using technology, we can improve cleanliness, reduce health risks, and contribute to a more eco-friendly and efficient waste management system.

Our motivation comes from a desire to create something meaningful and impactful. We wanted to build a system that not only works but also helps society in a small yet significant way. By minimizing manual effort and introducing real-time monitoring and alert systems, we hope our project can be a stepping stone towards smarter, cleaner cities and a healthier environment.

1.2 OBJECTIVE

The main objective of this project is to design and develop an automated Automated Smart Waste Segregation System with Live Monitoring and Intelligent Bin Alerts that classifies waste into three categories i.e., wet, metallic, and plastic and directs it into the appropriate bin using sensor data and actuator control. The system should be capable of:

- Detecting the type of waste using moisture, metal sensors.
- Moving the waste via a conveyor belt and pausing it for detection using an ultrasonic sensor.
- Rotating a stepper-motor-based bin platform to position the correct bin.
- Displaying waste type and system status using an I2C LCD.
- Monitoring bin levels using ultrasonic sensors and displaying them on a mobile application.
- Sending real-time notifications through Firebase Cloud Messaging when any bin reaches near-full condition.
- Enabling emergency stopping of the system both physically and remotely via the app.

By meeting these objectives, the system aims to improve the efficiency of waste segregation, reduce manual labor, enable remote monitoring, and contribute to a more sustainable waste management process.

1.3 EXISTING SYSTEM

In the existing systems of waste management, segregation is predominantly carried out manually. Workers sort through mixed waste to identify and separate biodegradable, metallic, and plastic materials. This approach is inefficient, slow, and prone to error due to fatigue, inconsistency, or lack of awareness. Moreover, it poses serious health risks to workers who are often exposed to sharp objects, toxic materials, and infectious waste. In some automated systems, a single type of waste may be detected and separated using color sensors or basic metal detectors, but these lack versatility, accuracy, and cloud connectivity. Most existing solutions do not have any provision to monitor bin fill levels or notify concerned authorities when bins are full, leading to delayed clearance and unhygienic overflows.

Another major limitation is the absence of real-time data logging or remote accessibility. Most traditional systems require manual checks to verify bin status, resulting in inefficient

resource allocation and increased labor cost. The lack of emergency response features and real-time feedback also makes existing systems less reliable for large-scale implementation. Therefore, there is a critical need for a fully automated, intelligent, and IoT-enabled waste segregation system that not only identifies waste but also informs users about bin status and supports smart monitoring features.

1.4 PROPOSED SYSTEM

The proposed Automated Smart Waste Segregation System with Live Monitoring and Intelligent Bin Alerts is a fully automated and sensor-driven solution designed to overcome the limitations of existing manual and semi-automated models. The system begins by transporting waste via a motorized conveyor belt. At the detection point, three sensors—capacitive, inductive, and moisture—are used to determine the waste type. The conveyor halts temporarily, and the waste is sorted accordingly. A NEMA 17 stepper motor rotates a bin assembly to align the correct bin based on the sensor classification. Once positioned, the waste is directed into the respective bin. This precise mechanism ensures high-speed, accurate sorting without manual intervention.

Each bin is equipped with an ultrasonic level sensor that continuously checks how full the bin is. These readings are transmitted to a Firebase database through a NodeMCU. The data is displayed in a custom-built mobile application developed using Flutter. When any bin exceeds a certain threshold, the app sends out real-time push notifications using Firebase Cloud Messaging. A buzzer is also activated to provide a local alert. An emergency button is implemented on the system for safety, and the same functionality is available digitally in the mobile app. An I2C LCD module is used to display current operation and detected waste type on-site.

Features of the Proposed System:

- Reduced human intervention.
- Sensor-based identification of biodegradable, plastic, and metal waste.
- Conveyor mechanism for efficient transport of waste items.
- Real-time monitoring of bin fill levels.
- Mobile app connectivity for remote monitoring.
- Firebase-based notification system when bins are full.
- Emergency stop switch (physical and digital).

- Real-time display of system status using LCD.

This system not only improves waste management efficiency but also reduces dependency on human labor, introduces real-time monitoring, and supports scalability for use in public and industrial settings.

1.5 ORGANIZATION OF PROJECT

The project can be organized as follows:

- **Chapter 1** introduces the project, highlighting the motivation, objectives, current limitations, and the proposed solution.
- **Chapter 2** provides a detailed literature survey that explores similar existing systems and relevant research studies in automation, IoT, and waste management.
- **Chapter 3** covers the hardware components used, including sensors, microcontrollers, motor drivers, and actuators, along with their technical specifications.
- **Chapter 4** explains the software requirements, coding platforms, libraries used, and Firebase integration steps for real-time data handling.
- **Chapter 5** elaborates on the design and implementation of the system, including block diagrams, wiring explanations, and working flow.
- **Chapter 6** presents the results of the implemented system with prototype images, test outputs, and real-time functioning validation.
- **Chapter 7** concludes the project with key findings and discusses the future scope for expanding the system with more features and scalability options.

CHAPTER 2

LITERATURE SURVEY

This chapter is set out to provide the overall analysis of various research approaches deployed and several options and achievements attained by different writers on our specified field of study and applications. The content here will take account of concepts of various researchers that have already made extensive research on the tools and applications with their collective efforts which has backed to the growth of technology.

Krishnapriya S. et al. [1] (2024) proposed an IoT-enabled waste segregation and monitoring system that uses IR, inductive proximity, moisture, and ultrasonic sensors to classify waste as dry, wet, or metallic. The system operates on an Arduino microcontroller and uses a Wi-Fi module for cloud connectivity. However, this system lacks real-time mobile notifications and does not support app integration. It is limited to basic online monitoring via a cloud dashboard and does not implement automated mechanical sorting.

Riyaj Multani et al. [2] (2024) developed an automatic waste segregation and monitoring system that classifies waste using ultrasonic sensors and basic conditional logic. The system transmits sensor data to a local interface but does not utilize cloud services or mobile apps. It is mainly designed for local monitoring with no provision for real-time alerts or emergency handling. The solution is primarily suited for small-scale domestic usage.

John V. et al. [3] (2023) proposed a simple waste segregation model using three basic sensors—moisture, metal, and IR—to separate waste. The controller used was Arduino Uno, and the system operated locally with no IoT connectivity or notification features. It focused purely on physical segregation and lacked any real-time communication or app-based monitoring.

Dewi Fitriani et al. [4] (2023) designed a smart waste bin using Wemos D1 microcontroller that opens and closes automatically based on ultrasonic distance detection. The system sends the bin status (empty, half, full) using Firebase, but only to a basic Android application. It lacks waste type segregation or sensor-based classification and does not provide push notifications or automation control.

Raj S. et al. [5] (2023) presented an IoT-based waste bin monitoring system that

detects bin fill levels using ultrasonic sensors and transmits the data to a local server via Wi-Fi. However, it does not segregate waste nor includes mobile notifications or a user app interface. It is more of a status-monitoring project rather than an active waste management solution.

By studying these works, we observed that most existing systems are limited either to basic waste level detection or simple categorization without automation, real-time alerts, or mobile application control. Our project stands out by integrating sensor-based waste classification, a rotating bin mechanism, bin fill level monitoring, and Firebase-powered mobile notifications, all managed under one smart embedded IoT system.

CHAPTER 3

HARDWARE COMPONENTS

3.1 Arduino Mega

The Arduino Mega is a microcontroller board developed by **Arduino.cc**. It is based on the ATmega2560 microcontroller chip, which offers enhanced memory and I/O capabilities compared to other Arduino boards. The Arduino Mega is a cost-effective and powerful development board with a larger form factor, suitable for complex projects such as robotics, automation systems**,** and IoT applications.

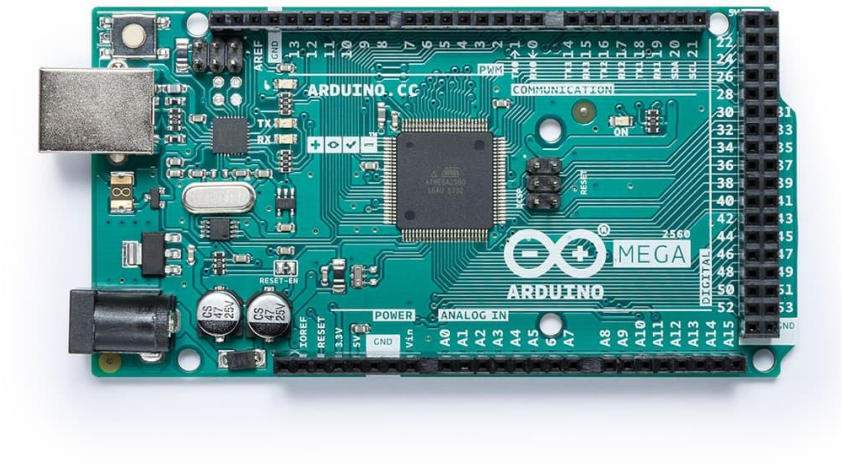


FIG 3.1 Arduino Mega

The Arduino Mega features a 16 MHz clock speed, 256 KB of flash memory, 8 KB of SRAM, and 4 KB of EEPROM. It also provides an extensive set of I/O options, including 54 digital input/output pins (of which 15 can be used as PWM outputs), 16 analog inputs, 4 UARTs (hardware serial ports), and SPI and I2C communication support.

One of the strengths of the Arduino Mega is its compatibility with Arduino IDE, supporting programming in C/C++ using a user-friendly environment. This makes it ideal for both beginners and advanced users to develop and deploy embedded systems effectively.

Unlike single-board computers like the Raspberry Pi, the Arduino Mega is a dedicated microcontroller. It does not run an operating system such as Linux. Instead, it is designed specifically for handling real-time control tasks such as controlling motors, reading sensor data, operating LEDs, and managing other components in embedded applications.

The Arduino Mega stands out as a low-cost, high-performance microcontroller board with a large number of digital and analog interfaces, making it an ideal choice for scalable and complex electronics projects.

Key features include:

- ATmega2560 microcontroller chip developed by Atmel (now Microchip Technology).
- 16 MHz clock speed for reliable real-time performance.
- 256 KB of flash memory for storing code (8 KB used by bootloader).
- 8 KB of SRAM and 4 KB of EEPROM for dynamic and persistent data storage.
- USB connection for programming and serial communication with the PC.
- 54 × digital I/O pins (15 capable of PWM output) for versatile interfacing.
- 16 × analog input pins with 10-bit resolution for sensor data acquisition.
- 4 × UART (hardware serial ports) for reliable serial communication.
- Supports SPI and I2C communication protocols.
- Integrated voltage regulators: 5V and 3.3V outputs available.
- Open-source hardware and software support via Arduino IDE.
- Large number of GPIO pins makes it ideal for complex, multi-sensor, and multi-module embedded systems.

Arduino Mega pinout:

The Arduino Mega 2560 is based on the ATmega2560 microcontroller and is designed to offer a significantly larger number of I/O pins and peripheral features compared to standard Arduino boards. Its extensive pin availability and memory make it highly suitable for

projects involving multiple sensors, actuators, or complex operations like robotics and embedded systems.

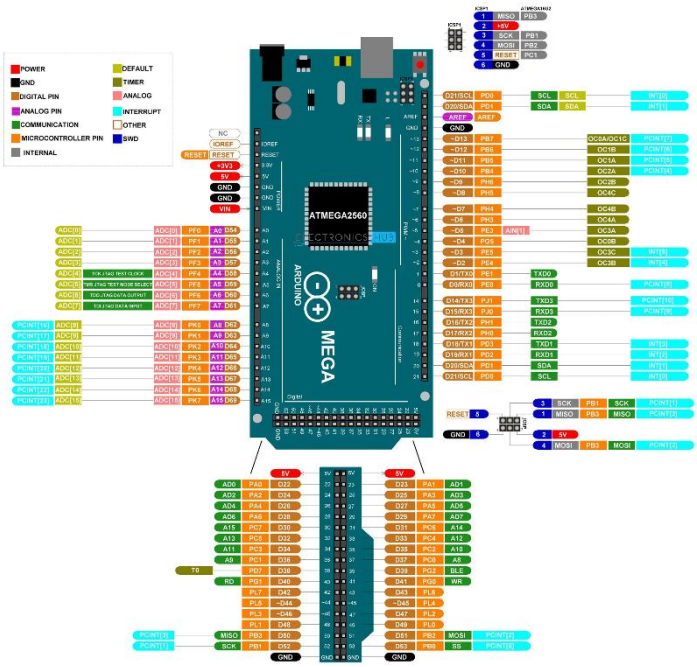


FIG 3.2 The pinout of the Arduino Mega Board

The Mega 2560 offers 54 digital I/O pins, 16 analog input pins, and 4 UART serial communication ports, making it ideal for multitasking applications. The pinout has been structured to bring out maximum flexibility in using various features like SPI, I2C, PWM, external interrupts, and analog-to-digital conversion.

The board operates at 5V logic level, and can be powered either via USB or through an external power supply using the DC barrel jack or the VIN pin. The operating voltage range for external input is typically between 7V to 12V, although the board can tolerate up to 20V in exceptional cases (not recommended for long-term use).

Key Functional Pin Groups:

- **Digital I/O (Pins 0 to 53):** These can be configured as either input or output. Among them, 15 pins support PWM output, marked with a tilde (~) such as ~2, ~3, ~4, etc.
- **Analog Inputs (Pins A0 to A15):** These pins provide 10-bit ADC resolution and are primarily used for reading analog sensors. They can also be used as general-purpose digital I/O.
- **UART Ports:** The Mega has four serial ports for communication:

- Serial0: RX0 (Pin 0), TX0 (Pin 1)
- Serial1: RX1 (Pin 19), TX1 (Pin 18)
- Serial2: RX2 (Pin 17), TX2 (Pin 16)
- Serial3: RX3 (Pin 15), TX3 (Pin 14)

- **SPI Interface:** Located on pins:
  - MISO: Pin 50
  - MOSI: Pin 51
  - SCK: Pin 52
  - SS: Pin 53

    SPI communication can also be accessed via the dedicated ICSP header.

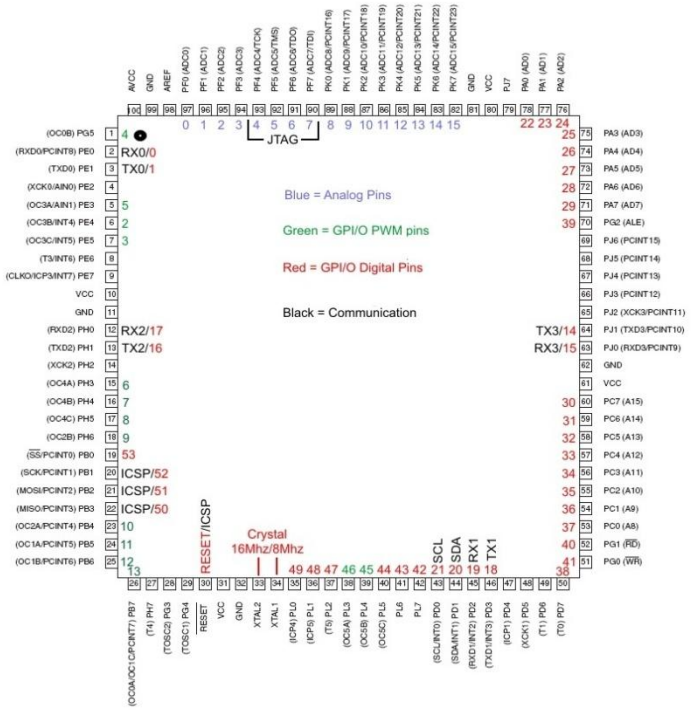


FIG 3.3 Pin diagram of Arduino Mega

- I**2C Interface:**
  - SDA: Pin 20
  - SCL: Pin 21

    These pins are used for I2C communication and are compatible with standard I2C libraries.

- Power **Pins:**

- **VIN:** The input voltage to the board from an external source.
- **5V:** Regulated power supply used to power the microcontroller and components.
- **3.3V:** A 3.3V supply derived from the onboard regulator (maximum ~50mA output).
- **GND:** Ground connections (multiple pins available).
- **IOREF:** Voltage reference for I/O pins.
- **RESET:** Active-low reset pin to restart the microcontroller manually.

Additional Features:

- **Crystal Oscillator:** The board has a 16 MHz quartz crystal for accurate timing.
- **USB Interface:** The ATmega16U2 microcontroller on the board handles USB-to-serial communication and can be reprogrammed as a USB device or HID.
- **ICSP Header:** This allows programming of the ATmega2560 microcontroller using an external programmer.
- **Auto Reset:** When connected to a computer via USB, the Mega 2560 can automatically reset and start code execution.

Mechanical specification:

The Arduino Mega 2560 is built on a standard two-sided PCB with a rectangular layout measuring 101.5mm × 53.3mm (4 inches × 2.1 inches). The board features a USB Type-B port located on the top left corner, used for programming and serial communication. Power input via a 2.1mm DC barrel jack is positioned on the left side of the board for external power supply.

The Mega 2560 follows the classic Arduino form factor, with mounting holes at each corner to allow easy fixing in enclosures or mechanical setups. The pin headers follow a 2.54mm (0.1") pitch compatible with standard female jumper wires, breadboards, and perfboards. There are 4 mounting holes spaced for stable mounting.

The board thickness is approximately 1.6mm, and the USB and barrel jack components protrude slightly above the board profile.

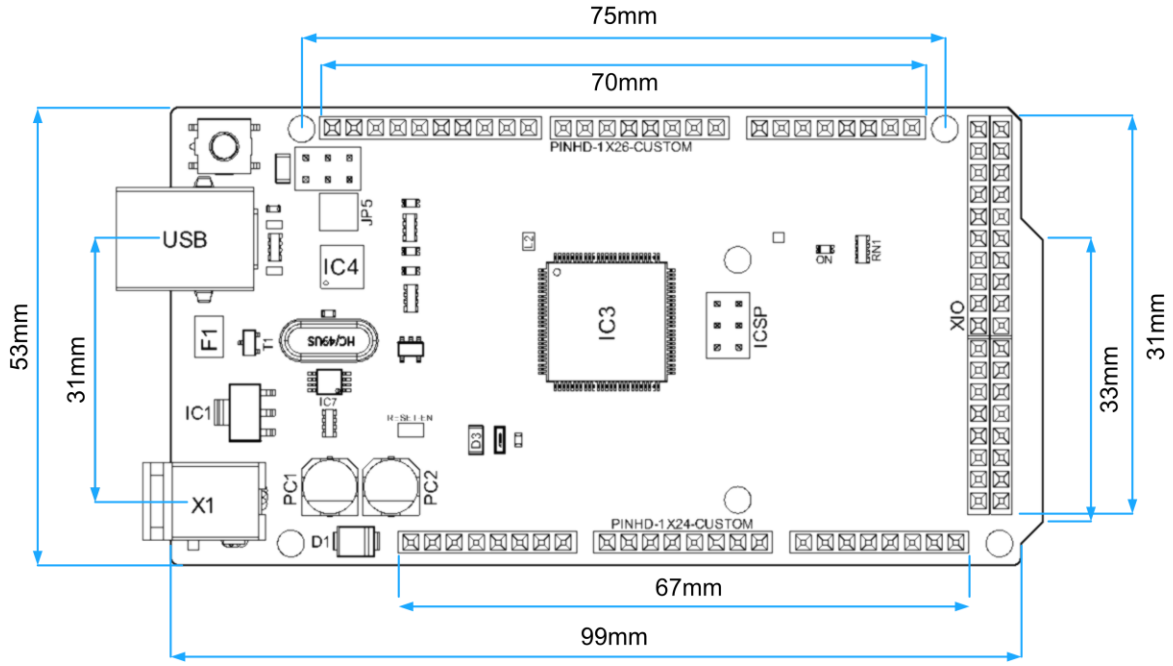


FIG 3.4 The dimensions of the Arduino Mega Board

Recommended operating conditions:

Operating Temp Min	-40°C
Operating Temp Max	+85°C (typical), up to 125°C for ATmega2560 chip
Input Voltage (recommended)	7V to 12V
Input Voltage (limits)	6V (min) – 20V (max)
Logic Level Voltage	5V
Analog Reference Voltage (AREF)	5V (default)
DC Current per I/O Pin	20 mA max
DC Current for 3.3V Pin	50 mA max

3.2 YL-69 Moisture sensor

The **YL-69** is a commonly used **soil moisture sensor** designed to measure the water content in soil. It consists of two main components: the **YL-69 sensor probe**, which detects moisture levels, and a **comparator module (usually an LM393)** that converts the analog signal into a digital output based on a predefined threshold.

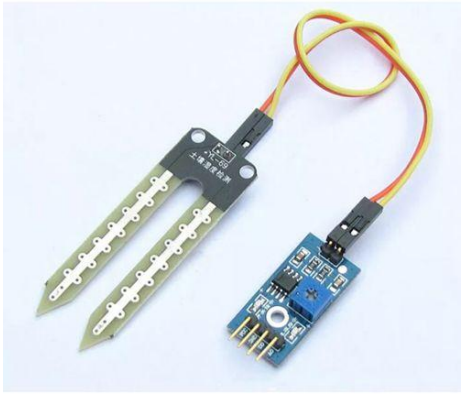


FIG 3.5 YL-69 Moisture sensor

The sensor works by measuring the resistance between two metallic probes inserted into the soil. When the soil is wet, it conducts electricity better, resulting in lower resistance and higher analog output. Conversely, when the soil is dry, resistance increases, and the analog output drops.

The sensor can be interfaced with any microcontroller, including Arduino and Raspberry Pi, using the analog pin (for precise readings) or the digital pin (to detect dry/wet status). A potentiometer on the module allows setting the threshold for digital output. This sensor is not factory calibrated, so user calibration is recommended for accurate results.

The connection diagram for this sensor is simple:

- VCC → 3.3V or 5V
- GND → Ground
- AO (Analog Output) → Analog input pin on microcontroller
- DO (Digital Output) → Digital input pin (optional)

The YL-69 is ideal for automatic irrigation systems, smart garden applications, and soil monitoring projects.

YL-69 Specifications

- **Operating Voltage:** 3.3V to 5V
- **Operating Current:** < 20mA
- **Output:** Analog and Digital (via comparator on module)
- **Sensor Type:** Resistive
- **Measurement Range:** Wet to dry (relative moisture level)

- **Analog Output:** Varies with soil moisture (higher voltage = wetter soil)
- **Digital Output:** LOW when soil moisture is above threshold, HIGH when below (adjustable via onboard potentiometer)
- **Interface:** 4-pin header (VCC, GND, A0, D0)
- **Dimensions:** Probe – approx. 60mm x 20mm
- **Accuracy:** Relative, not calibrated in absolute moisture %

3.3 Metal sensor

The metal detection sensor used in this project is a custom-built module developed using basic electronic components and a copper coil. The primary function of this sensor is to detect metallic objects on the conveyor belt as part of the waste classification process. When a metallic item passes over the sensor coil, it induces a change in electromagnetic properties, which can be interpreted by the connected microcontroller.

FIG 3.6 Metal sensor

Specifications:

Parameter	Specification
Sensor Type	Electromagnetic Metal Detector
Coil Diameter	~6–8 cm (approx., based on image)
Power Supply	5V DC (from microcontroller)
Output	Digital Signal (High/Low)
Detection Range	~1–3 cm (depending on object size)

Parameter	Specification
Response Time	Instantaneous (<1s)

3.4 HC SR04 Ultrasonic sensor

The **HC-SR04** is a widely used ultrasonic distance sensor capable of measuring distances from 2 cm to 400 cm with high accuracy. It uses sonar to determine distance to an object by emitting an ultrasonic pulse and measuring the time it takes for the echo to return. The sensor module consists of an ultrasonic transmitter, a receiver, and a control circuit. It provides digital I/O pins for easy interfacing with microcontrollers like Arduino, Raspberry Pi, etc. This sensor is commonly used in obstacle detection, robotics, and automation systems.

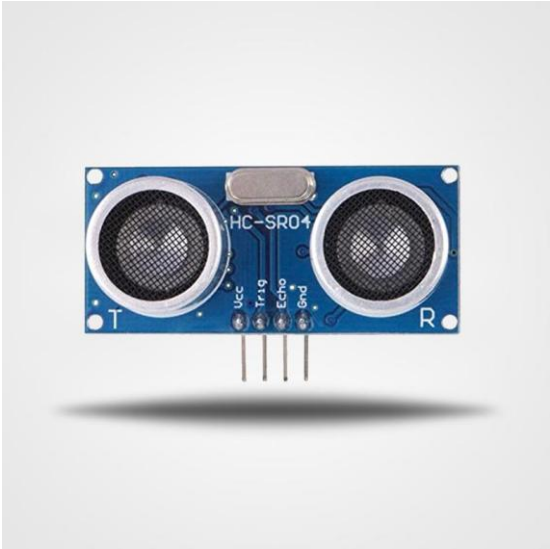


FIG 3.7 HC-SR04 Ultrasonic sensor

The HC-SR04 is easy to interface and requires only four pins: VCC, GND, TRIG (trigger input), and ECHO (echo output). When a HIGH pulse of 10µs is sent to the TRIG pin, the module emits an 8-cycle ultrasonic burst at 40kHz and waits for it to reflect back. The time it takes to receive the reflected signal on the ECHO pin is used to calculate the distance.

HC-SR04 Specifications

- **Operating Voltage:** 5V DC
- **Operating Current:** 15mA
- **Operating Frequency:** 40kHz

- **Measuring Range:** 2 cm to 400 cm (0.02 m to 4 m)
- **Measuring Angle:** 15°
- **Accuracy:** ±3 mm
- **Trigger Input Signal:** 10µs TTL pulse
- **Echo Output Signal:** TTL level signal, proportional to distance
- **Interface:** 4-pin (VCC, Trig, Echo, GND)
- **Dimensions:** 45mm × 20mm × 15mm

3.5 Conveyor belt

A conveyor belt is a continuous moving band used for transporting materials or objects from one place to another. In a Automated Smart Waste Segregation System with Live Monitoring and Intelligent Bin Alerts, a mini conveyor belt is typically used to carry waste materials across different sensing stations for classification and segregation. The conveyor is usually driven by a DC motor, allowing precise control over speed and direction using motor drivers like the L298N.

The conveyor plays a key role in automating the waste segregation process, as it provides consistent movement of waste for detection by sensors such as inductive, capacitive, moisture, and ultrasonic. It can be programmed to stop when an object is detected or divert waste to the appropriate bin based on sensor readings.

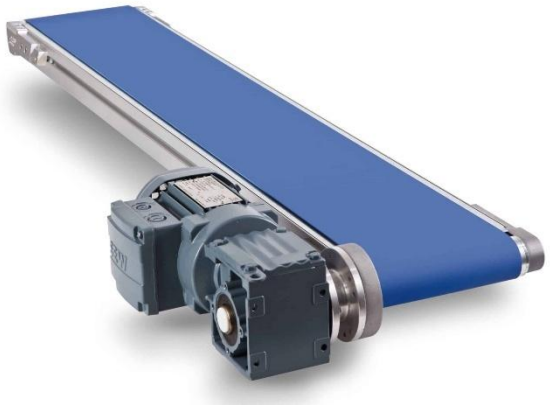


FIG 3.8 Conveyor belt

The belt is usually made from durable rubber or synthetic material and is mounted on rollers or pulleys to maintain tension. Depending on the setup, the motor can be powered by a 12V SMPS and controlled by an Arduino or other microcontroller.

Conveyor Belt Specifications

- **Motor Type:** DC motor or geared motor
- **Operating Voltage:** Typically, 6V–12V (commonly powered by 12V SMPS)
- **Motor Speed:** 100–300 RPM (depends on model)
- **Control Method:** L298N motor driver module (PWM control)
- **Direction Control:** Forward/Reverse (via microcontroller logic pins)
- **Load Capacity:** Light-weight objects (suitable for paper, plastic, metal waste)
- **Belt Material:** Rubber or synthetic fiber
- **Frame Material:** Plastic or aluminum for mini versions
- **Typical Dimensions:** Varies (e.g., 40cm x 10cm for mini versions)
- **Applications:** Waste segregation systems, mini-industrial automation, robotics, sorting mechanisms

3.6 Buzzer

Buzzer or beeper is an audio signalling device,[1] which may be mechanical, electromechanical, or piezoelectric (piezo for short). Typical uses of buzzers and beepers include alarm devices, timers, and confirmation of user input such as a mouse click or keystroke.

A buzzer is understood as a device that creates an audible tone under the influence of an applied external voltage. This output may either be in the form of a buzzing or a beeping sound. This is a result of the induced rapid movements created in the diaphragm of the buzzer.

The main function of this is to convert the signal from audio to sound. Generally, it is powered through DC voltage and used in timers, alarm devices, printers, alarms, computers, etc. Based on the various designs, it can generate different sounds like alarm, music, bell & siren.

FIG 3.9 Buzzer

Specifications

- The frequency range is 3,300Hz

- Operating Temperature ranges from – 20° C to +60°C
- Operating voltage ranges from 3V to 24V DC
- The sound pressure level is 85dBA or 10cm
- The supply current is below 15mA

3.7 DC MOTOR

A DC motor is an electrical motor that uses direct current (DC) to produce mechanical force. The most common types rely on magnetic forces produced by currents in the coils. Nearly all types of DC motors have some internal mechanism, either electromechanical or electronic, to periodically change the direction of current in part of the motor. A DC motor is any of a class of rotary electrical machines that converts direct current electrical energy into mechanical energy. The most common types rely on the forces produced by induced magnetic fields due to flowing current in the coil. Nearly all types of DC motors have some internal mechanism, either electromechanical or electronic, to periodically change the direction of current in part of the motor.

FIG 3.10 DC Motor

DC motors were the first form of motors widely used, as they could be powered from existing direct-current lighting power distribution systems. A DC motor's speed can be controlled over a wide range, using either a variable supply voltage or by changing the strength of current in its field windings. Small DC motors are used in tools, toys, and appliances. The universal motor, a lightweight brushed motor used for portable power

tools and appliances can operate on direct current and alternating current. Larger DC motors are currently used in propulsion of electric vehicles, elevator and hoists, and in drives for steel rolling mills. The advent of power electronics has made replacement of DC motors with AC motors possible in many applications.

3.8 I2C LCD DISPLAY

The I2C version of the 16×2 LCD is widely used in embedded and automation projects due to its simplicity in wiring and reduced pin usage. Instead of using multiple data pins and control pins like the parallel 16×2 LCD, this module uses only two communication lines (SDA and SCL) thanks to the built-in I2C adapter on the back of the display.

This significantly reduces the complexity of connections and conserves GPIO pins, especially when using microcontrollers with limited I/O like the Arduino Uno or ESP8266. It still displays 16 characters on 2 lines (16×2), with each character made up of a 5×8 pixel matrix.

FIG 3.11 I2C LCD Display

Pin Description:

- **GND** – Ground pin connected to the GND of the microcontroller.

- **VCC** – Power supply pin connected to 5V from the microcontroller or power source.
- **SDA (Serial Data)** – This is the data line for I2C communication. Connect this to the SDA pin of the microcontroller (A4 on Arduino Uno).
- **SCL (Serial Clock)** – This is the clock line for I2C communication. Connect this to the SCL pin of the microcontroller (A5 on Arduino Uno).

Advantages over regular 16×2 LCD:

- Requires only 2 data pins (SDA & SCL), reducing wiring.
- Built-in potentiometer on the I2C module helps adjust contrast.
- Built-in pull-up resistors (in some modules) for reliable I2C communication.
- Uses I2C protocol (address usually **0x27** or **0x3F**, can be scanned using an I2C scanner sketch).

3.9 L298N DC Motor driver

The L298N is one of the most commonly used dual H-Bridge motor driver ICs, capable of controlling the direction and speed of two DC motors simultaneously. It is ideal for robotic and automation projects where motor control is required. The module is based on the L298N dual full-bridge driver IC, which can drive inductive loads such as relays, solenoids, DC motors, and stepper motors.

This module simplifies motor control using PWM (Pulse Width Modulation) signals and provides an onboard 5V regulator to power logic circuitry. It supports motor voltages from 5V to 35V and logic levels of 5V, making it compatible with microcontrollers like Arduino, NodeMCU, ESP32, and Raspberry Pi.

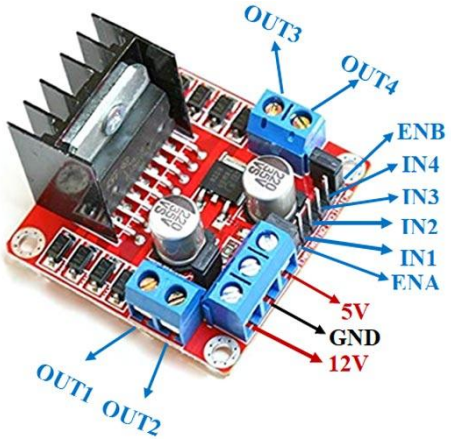


FIG 3.12 L298N Motor driver

Pin Description:

- **IN1 & IN2** – Used to control Motor A direction.
- **IN3 & IN4** – Used to control Motor B direction.
- **ENA** – Enables PWM speed control for Motor A.
- **ENB** – Enables PWM speed control for Motor B.
- **OUT1 & OUT2** – Connected to Motor A terminals.
- **OUT3 & OUT4** – Connected to Motor B terminals.
- **12V (VMS)** – Motor power supply (up to 35V).
- **5V** – 5V logic supply (used if onboard regulator is disabled).
- **GND** – Common ground.
- **Jumpers** – Usually used to enable internal 5V regulator or set ENA/ENB to always on.

Specifications:

- **Operating Voltage (Motor):** 5V – 35V
- **Logic Voltage (Vss):** 5V
- **Maximum Current:** 2A per channel (peak: 3A with good heat dissipation)
- **Number of Motors Controlled:** 2 DC motors or 1 stepper motor
- **PWM Control:** Yes (0–100% duty cycle)
- **Onboard Voltage Regulator:** 5V (can power microcontroller if motor voltage >7V)
- **Dimensions:** ~43mm x 43mm x 27mm
- **Protection:** Heat sink for thermal dissipation; no reverse polarity protection

3.10 BALL BEARINGS

Ball bearings are a type of rolling-element bearing that uses balls to maintain the separation between the bearing races. Their primary function is to reduce rotational friction and support radial and axial loads. They are widely used in machines and mechanical systems to enable smooth, efficient, and low-friction rotation of moving parts.

Ball bearings are commonly used in applications such as motors, fans, wheels, conveyor systems, and various rotating mechanisms in automation and robotics. Their design allows them to operate at high speeds and with minimal maintenance.

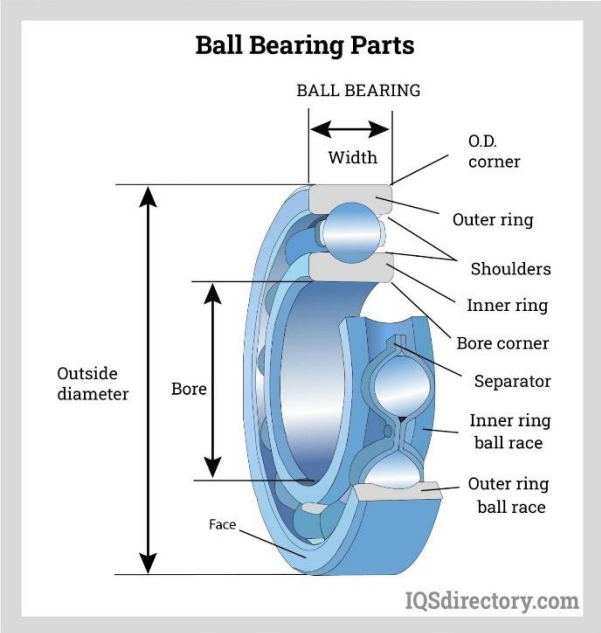
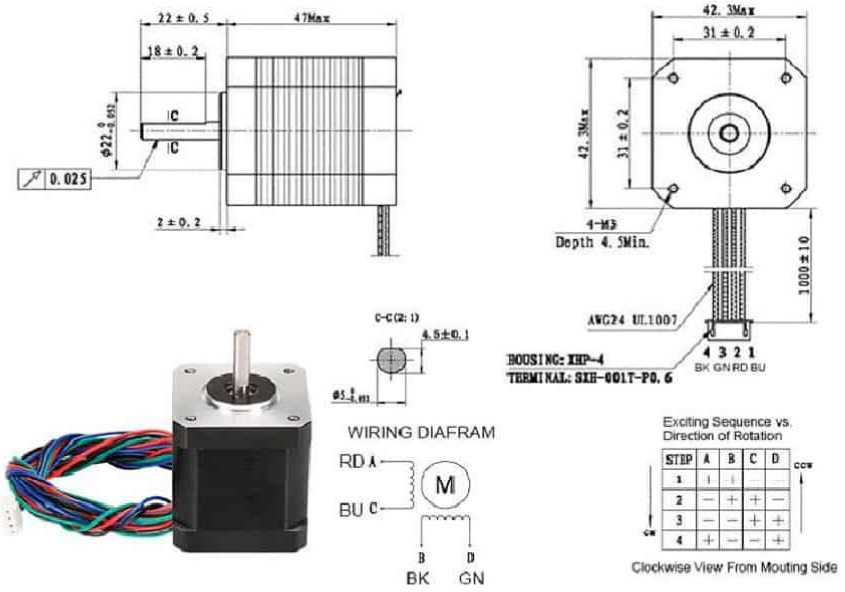


FIG 3.13 Ball bearings

3.11 NEMA 17 Stepper motor

NEMA 17 is a standard size for stepper motors defined by the National Electrical Manufacturers Association (NEMA). The "17" refers to the size of the faceplate — 1.7 inches (43.2 mm). NEMA 17 stepper motors are widely used in CNC machines, 3D printers, robotics, automation systems, and various mechatronics projects due to their precision, torque, and ease of control.

Unlike DC motors, stepper motors rotate in discrete steps, allowing for precise position and speed control without the need for feedback systems (open-loop control).



Motor Length (mm)	Step Angle (°)	Phase (NO.)	Rated Voltage (V)	Rated Current (A)	Resistance / Phase (Ω)	Inductance /Phase (mH)	Holding Torque (mN.m)	Detent Torque (mN.m)	Rotor Intertia (g.cm²)	Weight (kg)
47	1.8	2	2.6	2.0	1.3	1.9	480	24	82	0.36

FIG 3.14 NEMA17 Stepper motor

Specifications:

- **Step Angle:** 1.8° per step (200 steps per revolution)
- **Holding Torque:** 40 N·cm to 52 N·cm (approx. 4–5.2 kg·cm)
- **Rated Voltage:** 2.8V – 5V (depends on model)
- **Rated Current:** 1.2A to 2.0A per phase
- **Number of Phases:** 2
- **Number of Wires:** 4 (Bipolar) or 6 (Unipolar)
- **Shaft Diameter:** 5 mm (typically D-shaft or round)
- **Motor Size:** 42 mm x 42 mm (1.7 inches square faceplate)
- **Body Length:** Varies (typically 34 mm to 48 mm)
- **Max Operating Temperature:** Up to 80°C
- **Inductance:** 2.5 mH to 4.5 mH
- **Resistance:** ~1.2Ω to 2.0Ω per phase

3.12 A4988 Stepper motor driver

The A4988 is a popular microstepping motor driver developed by Allegro Microsystems, widely used to control bipolar stepper motors such as the NEMA 17. It allows precise control of motor speed and rotation through step and direction pins, making it ideal for 3D printers, CNC machines, robotic arms, and other automation projects.

The A4988 supports full-step, half-step, quarter-step, eighth-step, and sixteenth-step microstepping, allowing for smoother and quieter motor operation. It is typically used with microcontrollers like Arduino, Raspberry Pi, or any platform that supports digital I/O.

The driver is mounted on a breakout board with necessary protection and connectors, including current limiting, thermal shutdown, and short-circuit protection features.

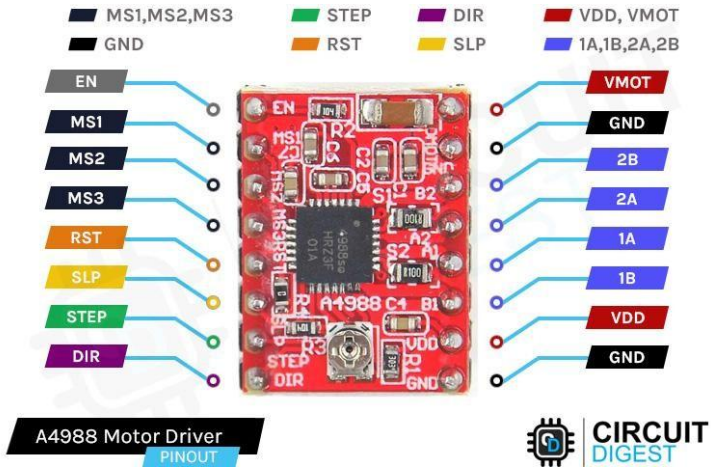


FIG 3.15 A4988 Stepper motor driver

Pin Description:

- **VMOT** – Motor supply voltage (8–35V)
- **GND** – Ground
- **VDD** – Logic voltage (3V–5.5V)
- **STEP** – Receives step signals from microcontroller
- **DIR** – Controls the direction of motor rotation
- **ENABLE** – Enables or disables the driver output (active LOW)
- **MS1, MS2, MS3** – Microstepping selection pins
- **RESET & SLEEP** – Used to reset or put the driver into low-power mode
- **OUT1A, OUT1B / OUT2A, OUT2B** – Connect to stepper motor coil wires

Specifications:

- **Motor Type Supported:** Bipolar Stepper Motors
- **Operating Voltage (Motor - VMOT):** 8V to 35V
- **Logic Voltage (VDD):** 3V to 5.5V
- **Max Motor Current:** Up to 2A per coil (with heat sink and cooling)
- **Microstepping Modes:** Full, 1/2, 1/4, 1/8, 1/16
- **Step Resolution (with MS1–MS3):** Configurable via jumpers or code
- **Current Control:** Adjustable via onboard potentiometer
- **Protection Features:**
    - Over-temperature shutdown
    - Under-voltage lockout
    - Crossover-current protection
    - Short-to-ground protection
- **Size:** Approx. 15mm × 20mm (very compact)
- **Heat Dissipation:** Requires a heat sink for high current usage

3.13 MG995 Servo motor

The MG995 is a high-torque, metal-geared servo motor widely used in robotic arms, steering mechanisms in RC vehicles, and other projects that require precise angular movement and high strength. It is a standard-size servo with strong metal gears, making it durable and reliable even under mechanical stress.

Unlike stepper motors, servo motors are designed for position control. The MG995 can rotate to a specific angle between 0° and 180° based on the PWM signal received. It is controlled using a single signal wire and requires no external motor driver.

The MG995 features a robust torque output, fast response, and good holding power, making it ideal for applications where accurate movement and high load capacity are essential.

FIG 3.16 MG995 Servo motor

Pin Description:

- **Brown (GND):** Connect to Ground of power supply/microcontroller
- **Red (VCC):** Connect to +5V or 4.8V–7.2V power supply
- **Orange (Signal):** Connect to PWM signal from microcontroller (e.g., Arduino digital pin)

Specifications:

- **Operating Voltage:** 4.8V to 7.2V
- **Stall Torque:**
  - 9.4 kg·cm @ 4.8V
  - 11 kg·cm @ 6.0V
- **Speed:**
  - 0.20 sec/60° at 4.8V
  - 0.16 sec/60° at 6.0V
- **Rotation Range:** 0° to 180° (limited by internal stops)
- **Gear Type:** Full Metal Gears
- **Motor Type:** DC motor with feedback potentiometer
- **Control Signal:** PWM (typically 50 Hz, pulse width 1–2 ms)
- **Dead Band Width:** ~5 µs
- **Dimensions:** 40.7 mm × 19.7 mm × 42.9 mm
- **Weight:** ~55g
- **Connector Type:** 3-pin female servo connector (compatible with standard RC receivers)

3.14 ESP8266

The ESP8266 NodeMCU is a low-cost, open-source IoT platform based on the ESP8266 Wi-Fi module developed by Espressif Systems. It is widely used in IoT and wireless-based embedded projects due to its built-in Wi-Fi, compact size, and full support for programming via Arduino IDE.

The board includes the ESP-12E module (based on the ESP8266 SoC), a USB-to-serial converter, voltage regulator, and a set of GPIO pins that allow easy integration with sensors, actuators, and other modules. NodeMCU also supports Lua scripting but is most commonly programmed using Arduino C/C++ for better flexibility and ease of use.

Its built-in Wi-Fi capabilities eliminate the need for external Wi-Fi modules, making it ideal for applications such as smart home systems, real-time sensor data logging, wireless control, and cloud communication using platforms like Firebase or MQTT.

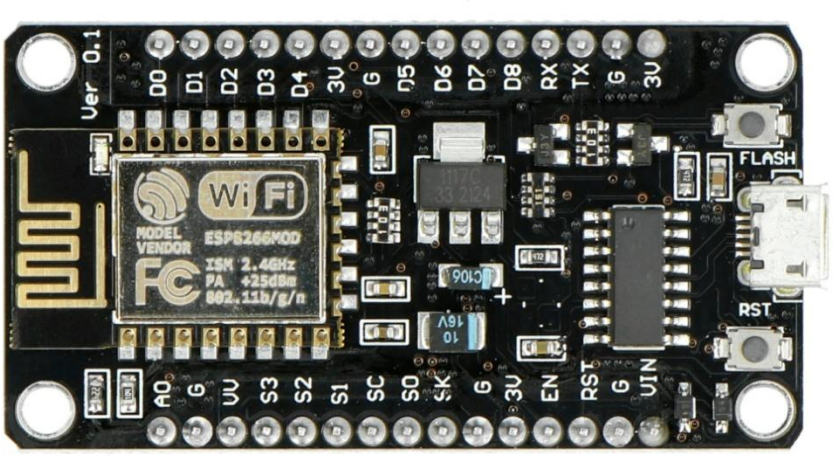


FIG 3.17 ESP8266

Key Features:

- **Microcontroller:** Tensilica L106 32-bit RISC processor (ESP8266EX)
- **Clock Speed:** 80 MHz (default), up to 160 MHz
- **Flash Memory:** 4 MB
- **SRAM:** 64 KB instruction + 96 KB data
- **Operating Voltage:** 3.3V (Do not apply 5V directly to GPIO)

- **Digital I/O Pins:** 11 GPIO pins (some with dual functionality)
- **Analog Input:** 1 ADC (10-bit resolution)
- **Communication Interfaces:** UART, SPI, I2C, PWM
- **Built-in Wi-Fi:** IEEE 802.11 b/g/n (2.4 GHz)
- **USB Interface:** Micro USB (for power and programming)
- **Programming Support:** Arduino IDE, Lua, MicroPython

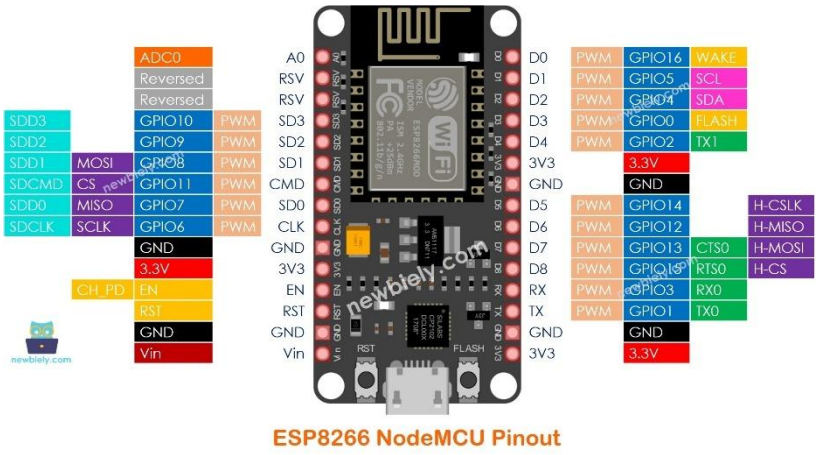


FIG 3.18 Pinout of ESP8266

The NodeMCU module typically provides 11 digital I/O pins, 1 analog input pin, and multiple communication interfaces such as UART, SPI, and I2C. The pinout is optimized for simplicity in IoT applications, minimizing wiring requirements with built-in USB connectivity and automatic reset circuitry.

The board operates at 3.3V logic level and can be powered either via the micro USB port or through the Vin pin using a regulated 5V source. Note that ESP8266 GPIOs are not 5V-tolerant, and supplying more than 3.3V to any GPIO can damage the board.

Key Functional Pin Groups:

- **Digital I/O (Pins D0 to D8):**
  These GPIO pins can be configured as input or output. Some support PWM, SPI, or I2C.
  - D0 (GPIO16) – Can be used for output but does not support interrupts
  - D1 (GPIO5), D2 (GPIO4), D5–D8 – Commonly used for I2C/SPI/PWM
- **Analog Input (A0):**

  - A0 is the only analog input pin available and supports 10-bit ADC resolution.
  - Voltage range: 0V to 1V (exceeding this may damage the pin)
- **UART Ports:**
  - **Serial (UART0):** TX (GPIO1), RX (GPIO3) – used for programming and debugging
  - Additional UARTs (UART1) exist but have limited functionality (TX only)
- **I2C Interface:**
  - SDA: D2 (GPIO4)
  - SCL: D1 (GPIO5)

    (I2C pins are not hardware-defined and can be configured in software)
- **SPI Interface:**
  - MOSI: D7 (GPIO13)
  - MISO: D6 (GPIO12)
  - SCK: D5 (GPIO14)
  - SS: D8 (GPIO15)

    (Used for interfacing displays, memory, etc.)

Power and Special Pins:

- **Vin:** Can be used to power the board with a regulated 5V supply
- **3V3:** Regulated 3.3V output (can supply sensors/modules)
- **GND:** Common ground
- **EN (Enable):** Must be pulled HIGH for the board to operate
- **RST:** Active-low reset input to restart the microcontroller
- **Flash:** Used internally or for manual flashing
- **ADC0 (A0):** Analog input (max 1.0V)

Recommended Operating Conditions:

Operating Voltage (Logic)	3.3V
Input Voltage (via USB)	5V
Digital I/O Voltage	3.3V (not 5V-tolerant)
Operating Temperature	-40°C to +125°C
Wi-Fi Frequency	2.4 GHz

3.15 12V 5A SMPS (Switched Mode Power Supply)

The **12V 5A SMPS** is a highly efficient and reliable power supply unit used to convert high voltage AC (typically 230V) into a stable 12V DC output. It is particularly suitable for electronic systems that require higher current loads, such as motors, drivers, sensors, and microcontrollers.

FIG 3.19 12V 5A SMPS (Switched Mode Power Supply)

Specifications:

- **Input Voltage**: 100V – 240V AC (50/60Hz)
- **Output Voltage**: 12V DC
- **Maximum Output Current**: 5 Amps
- **Maximum Output Power**: 60 Watts
- **Efficiency**: >85%
- **Output Type**: Regulated and stabilized DC
- **Protection Features**:
  - Overload Protection
  - Short Circuit Protection
  - Over-voltage Protection

CHAPTER 4

SOFTWARE REQUIREMENTS

4.1 ARDUINO SOFTWARE

The **Arduino Mega 2560** is a user-friendly and powerful microcontroller board used in both hobby and professional projects. The Arduino is open-source, which means hardware is reasonably priced and development software is free. This guide is for students in ME 2011, or students anywhere who are confronting the Arduino for the first time. For advanced Arduino users, prowl the web; there are lots of resources. The Arduino project was started in Italy to develop low-cost hardware for interaction design. An overview is on the Wikipedia entry for Arduino.

4.1.1 REQUIRED SPECIFICATIONS

- Arduino Mega 2560 development board
- USB programming cable (Type A to B)
- External power supply (7–12V recommended for standalone)
- Solderless breadboard and 22-gauge solid wire for prototyping
- Host PC with Arduino IDE (Windows/Mac/Linux)

The Arduino programming language is a simplified version of C/C++. If you know C, programming the Arduino will be familiar. If you do not know C, no need to worry as only a few commands are needed to perform useful functions. An important feature of the Arduino is that you can create a control program on the host PC, download it to the Arduino and it will run automatically. Remove the USB cable connection to the PC, and the program will still run from the top each time you push the reset button. Remove the battery and put the Arduino board in a closet for six months. When you reconnect the battery, the last program you stored will run. This means that you connect the board to the host PC to develop and debug your program, but once that is done, you no longer need the PC to run the program. Start the Arduino development environment. In Arduino-speak, programs are called “sketches”, but here we will just call them programs. In the editing window that comes up, enter the following program, paying attention to where semi- colons appear at the end of command line.

4.2 PROGRAM EXECUTION

Window will look something like this

- Click the upload button or Ctrl+U to compile the program and load on the Arduino Mega board.
- Click the serial monitor button . If all has gone well, the monitor window will show your message and look something like this program, paying attention to where semi-colons appear at the end of command lines.

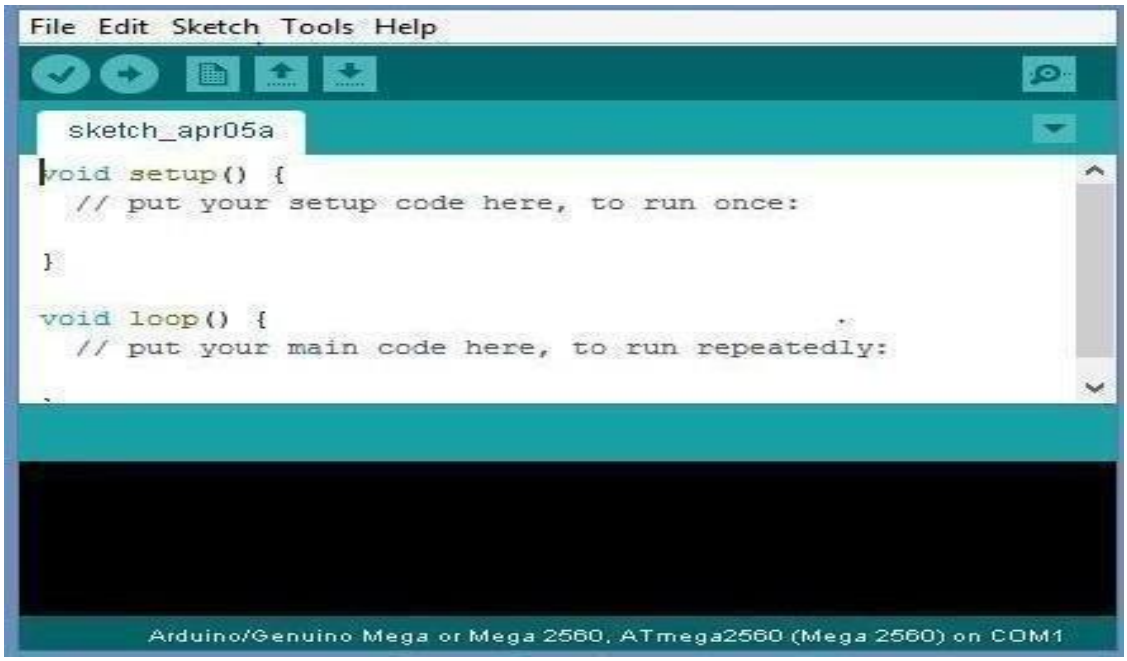


FIG 4.1

4.2.1 TROUBLE SHOOTING

If there is a syntax error in the program caused by a mistake in typing, an error message will appear in the bottom of the program window. Generally, staring at the error will reveal the problem. If you continue to have problems, try these ideas.

Run the Arduino program again

- Check that the USB cable is secure at both ends.
- Reboot your PC because sometimes the serial port can lock up
- If a "Serial port…already in use" error appears when uploading Next go to tick mark to verify the code
- Click on tool bar menu and then click on serial port

- Next select the port

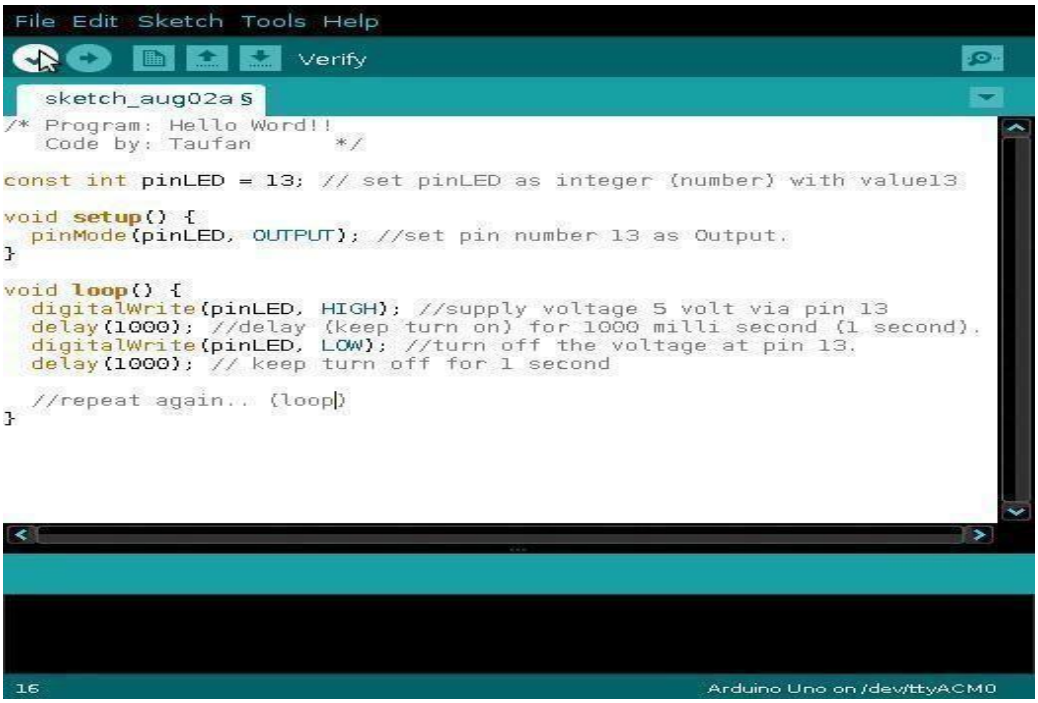


FIG 4.2

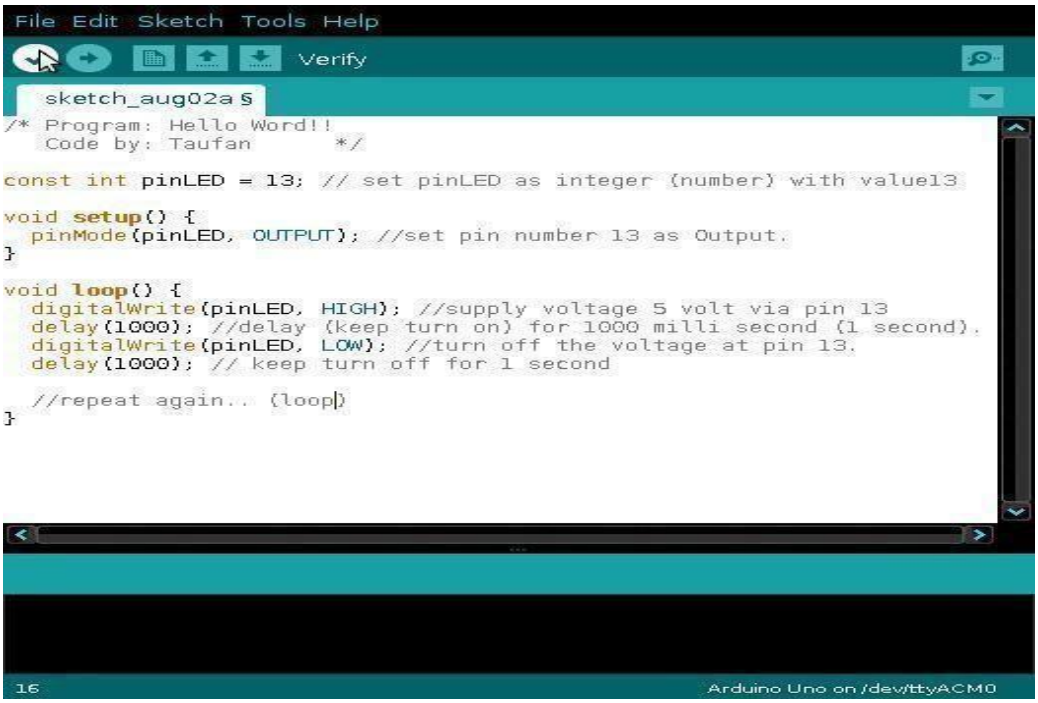


FIG 4.3

Now go to tool bar and click on board

- Select the board

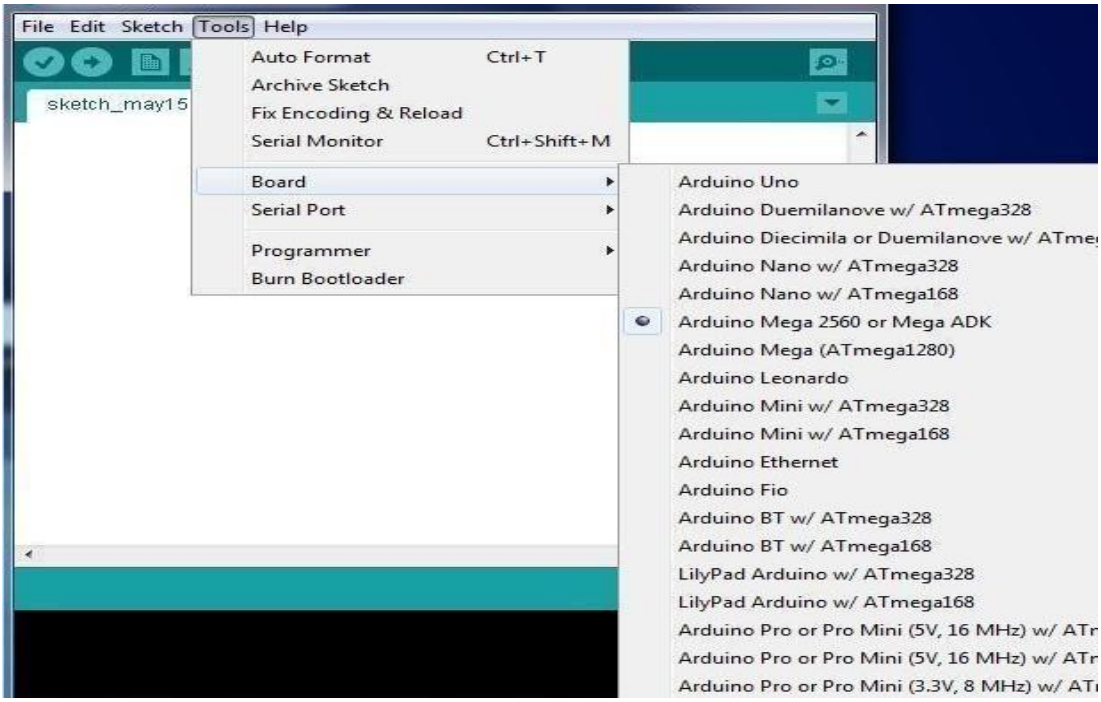


FIG 4.4

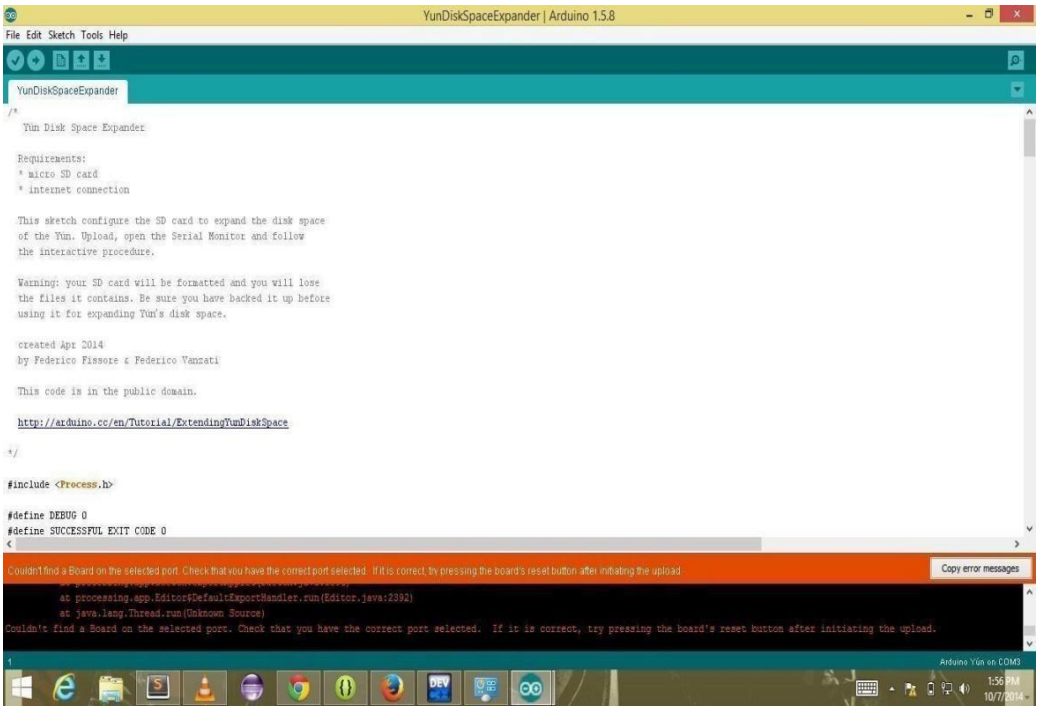


FIG 4.5

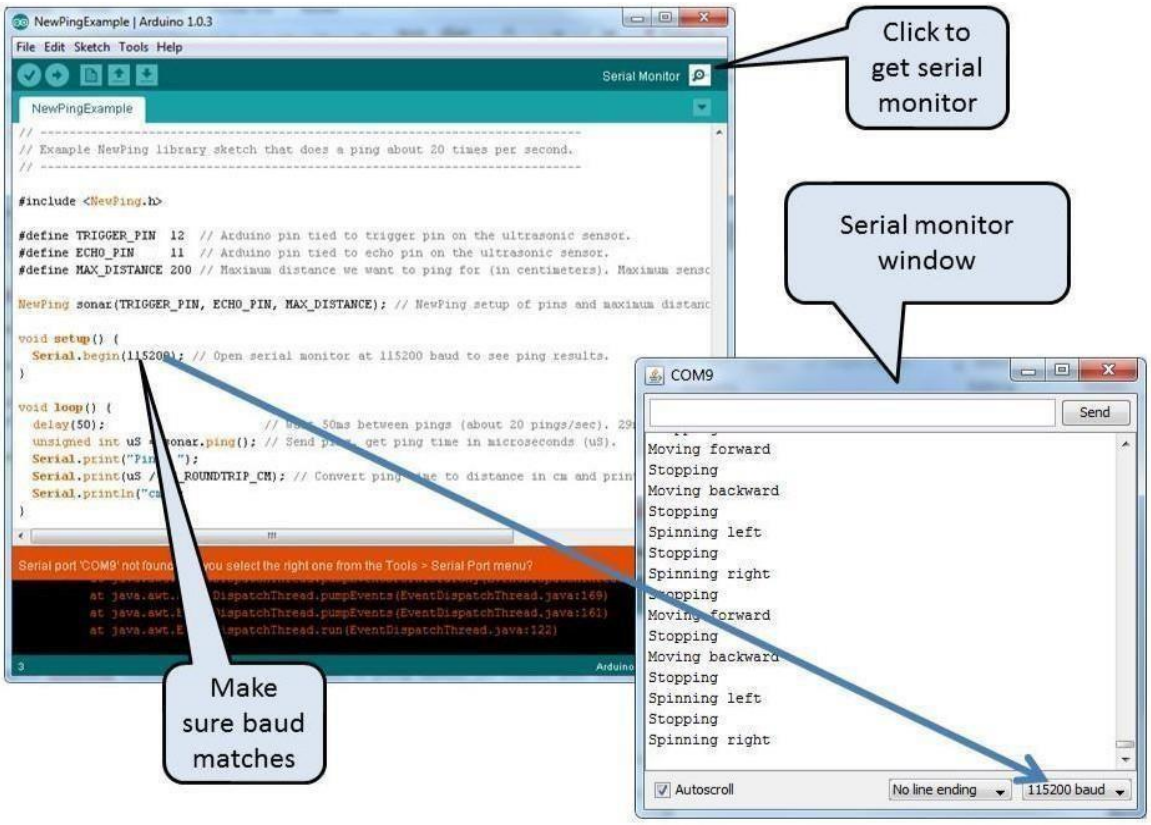

NewPingExample | Arduino 1.0.3
File Edit Sketch Tools Help
Serial Monitor
NewPingExample
Click to get serial monitor
Serial monitor window
Make sure baud matches
COM9
Send
Moving forward
Stopping
Moving backward
Stopping
Spinning left
Stopping
Spinning right
Stopping
Moving forward
Stopping
Moving backward
Stopping
Spinning left
Stopping
Spinning right
Autoscroll
No line ending
115200 baud

FIG 4.6

CHAPTER 5

DESIGN AND IMPLEMENTATION

5.1 BLOCK DIAGRAM

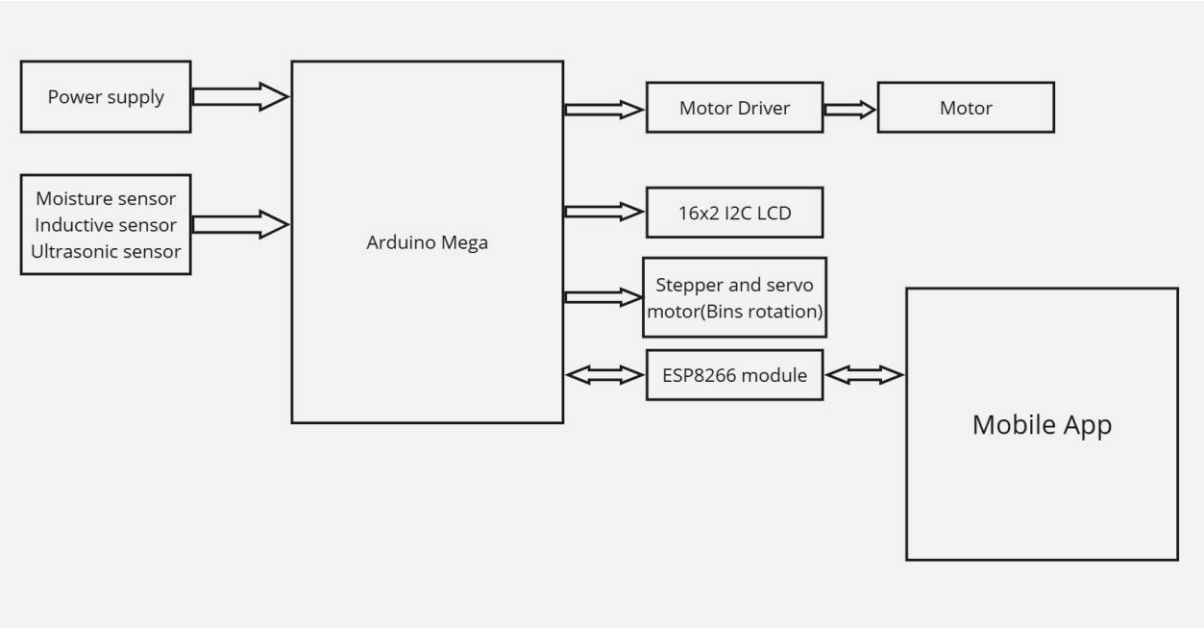


FIG 5.1 BLOCK DIAGRAM

- Upon powering on the system, the Arduino Mega initializes all connected components including the moisture sensor, inductive sensor, ultrasonic sensor, motor driver, ESP8266, and LCD.
- GPIO pins and communication protocols like I2C (for LCD) and UART (for ESP8266) are configured for communication.
- The Arduino continuously reads data from the moisture, inductive, and ultrasonic sensors at regular intervals.
- The moisture sensor detects if the waste is wet or dry.
- The inductive sensor checks for the presence of metallic content in the waste.
- The ultrasonic sensor is used to detect the distance of the waste and also monitors bin levels.
- Based on sensor data, the type of waste is determined and displayed on the 16x2 I2C LCD.
- The motor driver controls the conveyor DC motor, which moves the waste forward.
- When a waste item reaches the end, the system rotates the stepper or servo motor to

align the bin for that specific waste type.

- The ESP8266 module sends bin status data to the mobile app and also receives control commands like emergency stop.
- If a bin is nearly full, the system sends an alert to the user's phone via the mobile app, helping in timely disposal and preventing overflow.

5.2 Interfacing Buzzer with Arduino Mega

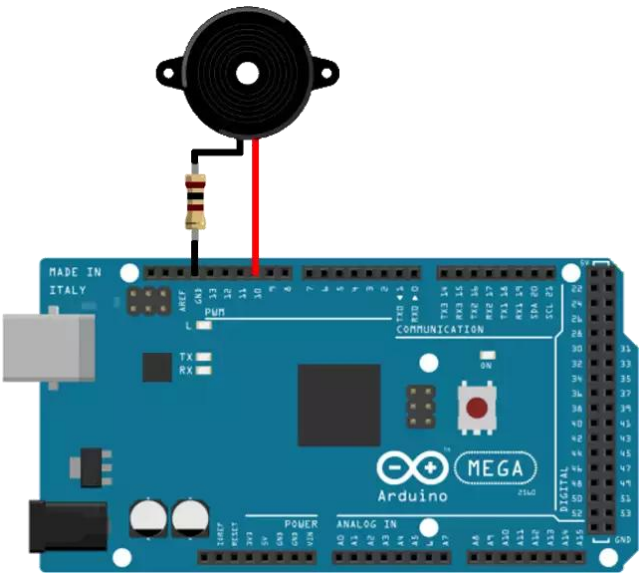


FIG 5.2

A Piezo buzzer is an electronic component used to produce sound, commonly used in embedded systems to give audio alerts or notifications. It is a two-terminal device, where the longer leg is the positive terminal. When voltage is applied, it emits a beep or tone.

In the proposed system, the buzzer is used to provide an alert when a bin is full or when an emergency stop condition is triggered. The Arduino Mega sends a HIGH signal to the buzzer through a digital I/O pin to activate it. The buzzer produces a short or continuous beep depending on how the signal is programmed.

To wire a buzzer to the Arduino Mega, connect the positive leg to digital pin 40 (or any other available digital pin) and the negative leg to the nearest GND pin.

5.3 Interfacing Sensors with Arduino Mega

The Arduino Mega 2560, with its vast number of GPIO pins (54 digital and 16 analog), is highly suitable for interfacing multiple sensors in parallel, making it ideal for complex systems like the Automated Smart Waste Segregation System with Live Monitoring and Intelligent Bin Alerts.

In the proposed setup, sensors like the moisture sensor, ultrasonic sensor, and a custom-made metal detection sensor using a capacitor are interfaced with the Arduino Mega. These sensors help detect different waste types and monitor the bin status effectively.

Connection Overview:

- The moisture sensor is connected to an analog input pin (e.g., A0) and is used to detect the presence of wet or biodegradable waste.
- The ultrasonic sensor is connected via digital pins (e.g., Trig → pin 6, Echo → pin 7) to measure the distance to objects and detect the fill level of the bins.
- The custom metal detection sensor, built using a capacitor-based circuit, has three pins:
    - Capacitor Discharge Pin → Connected to a digital output pin (e.g., pin 9)
    - Pulse Pin → Connected to a digital input pin (e.g., pin 10)
    - GND → Connected to the Arduino ground
      This sensor works by charging and discharging the capacitor and detecting changes in the pulse timing when a metal object is near.
- All sensor GND pins are connected to the GND of Arduino, and VCC pins (if applicable) are connected to 5V or 3.3V, depending on the sensor's requirements.

5.4 Interfacing DC Motor and L298N Driver Module with Arduino Mega

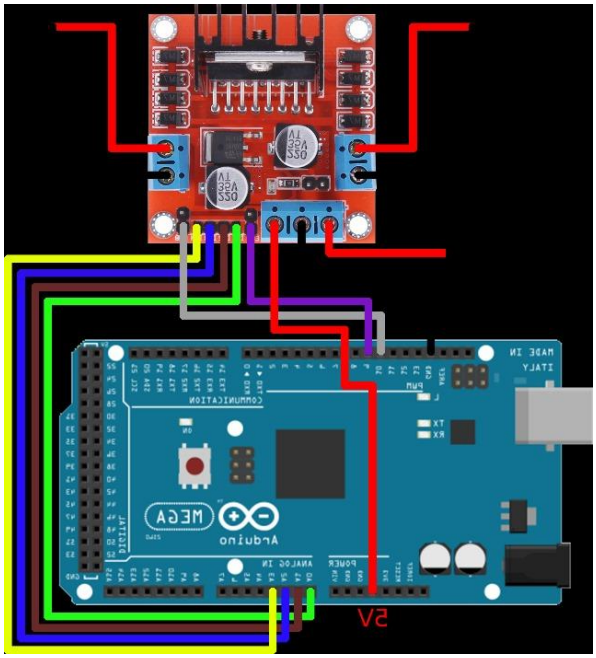


FIG 5.4 Interfacing L298N with Arduino Mega

The L298N Motor Driver Module is widely used to control DC motors and stepper motors in embedded and robotic systems. It can control both the direction and speed of two DC motors simultaneously using an H-bridge configuration. In the proposed system, the DC motor is used to drive the conveyor belt, which moves the waste toward the sensor section for segregation.

The Arduino Mega controls the DC motor through the L298N module by sending signals to the input pins (IN1, IN2) for direction control and a PWM signal to the ENA pin for speed control. The driver module is powered by a 12V power supply (such as a 12V SMPS), and it also powers the motor.

Wiring Connections:

- **L298N to Arduino Mega:**
  - **IN1** → Digital Pin 2
  - **IN2** → Digital Pin 3
  - **ENA** → Digital Pin 4 (PWM enabled for speed control)
  - **GND** → GND
  - **5V (optional)** → Not used if using onboard regulator
- **Motor Power Supply:**
  - **12V Input** → Connected to L298N **VMS** (motor power pin)

- **GND** → Common ground with Arduino

- **DC Motor Terminals:**
  - **OUT1** and **OUT2** → Connected to the two terminals of the DC motor

5.5 Interfacing NEMA 17 Stepper Motor with A4988 Driver and Arduino Mega

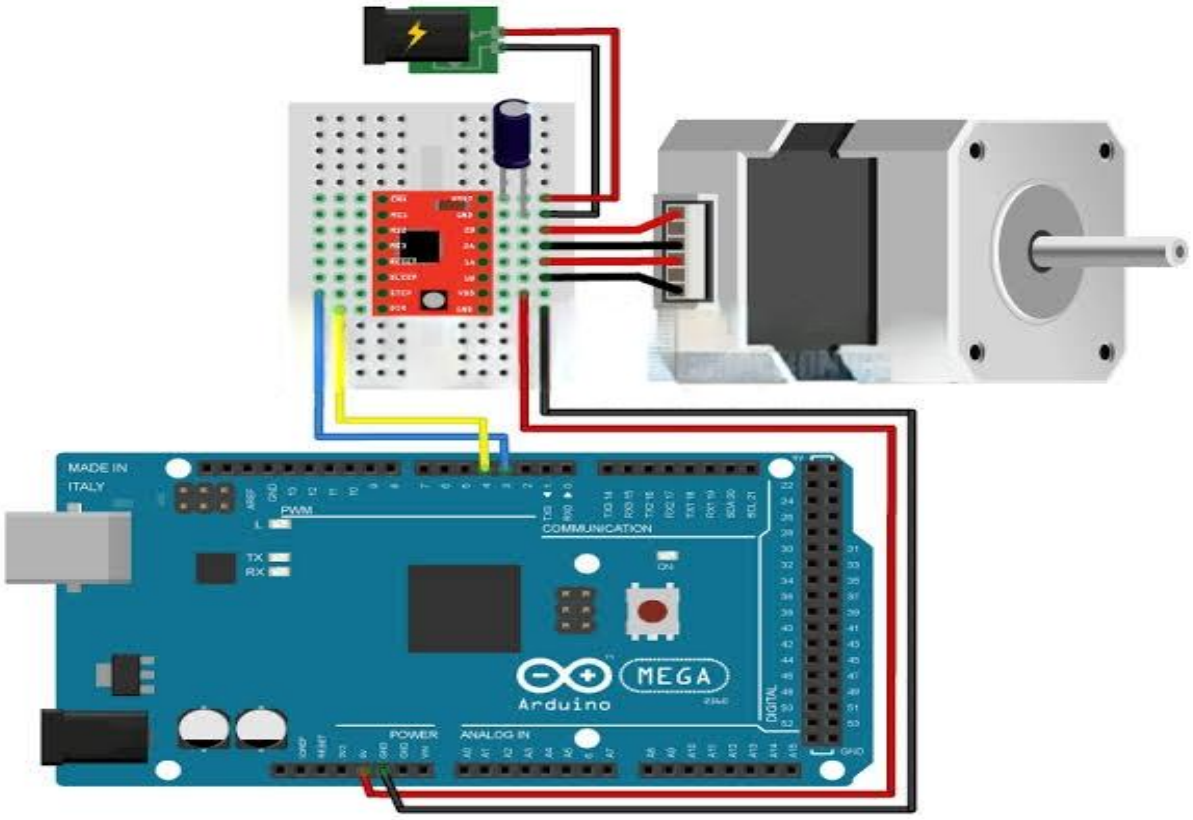


FIG 5.5 Interfacing NEMA 17 with A4988 and Arduino Mega

The NEMA 17 stepper motor is a precise and powerful motor widely used in applications requiring controlled rotation such as robotic arms, 3D printers, and waste sorting systems. In the proposed Automated Smart Waste Segregation System with Live Monitoring and Intelligent Bin Alerts, the NEMA 17 stepper motor is used to rotate the bin alignment mechanism, directing waste into the correct bin based on type.

To control the NEMA 17 motor, an A4988 stepper motor driver is used. This compact module is capable of driving a bipolar stepper motor using just two input signals from the Arduino Mega—STEP and DIR. The A4988 also supports microstepping, adjustable current control, and includes protections against overheating and short circuits.

Wiring Connections:

A4988 to Arduino Mega:

- **STEP** → Digital Pin 41
- **DIR** → Digital Pin 40
- **EN (Enable)** → Digital Pin 42 *(Optional, tie LOW to always enable)*
- **VDD** → 5V
- **GND** → GND
- **VMOT** → 12V (from external power supply for motor)
- **GND (Power GND)** → Common GND with Arduino and motor power

Stepper Motor to A4988:

- **2B, 2A, 1A, 1B** → Connect to the corresponding four wires of the NEMA 17 motor

5.6 Interfacing I2C LCD Display with Arduino Mega

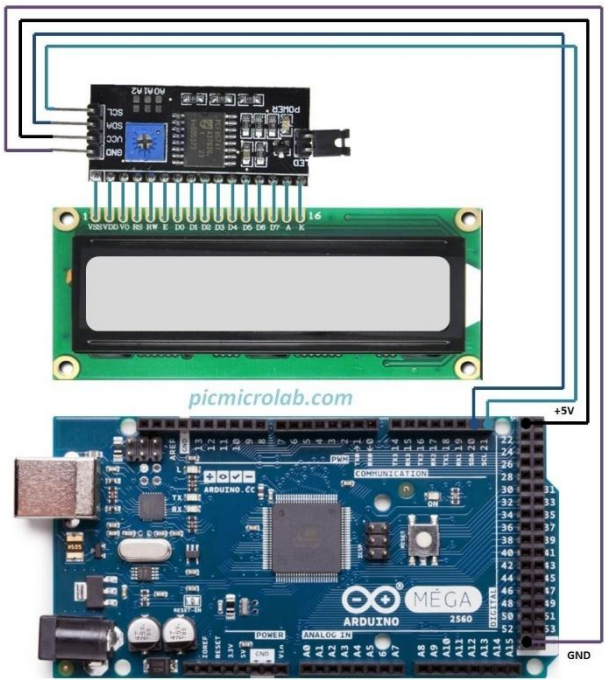


FIG 5.6 Interfacing I2C LCD display with Arduino Mega

The I2C LCD Display is a simplified version of the traditional 16×2 LCD that uses the I2C communication protocol to reduce the number of pins required for interfacing. Instead of 6–8 connections like the standard parallel LCD, the I2C module requires only two wires (SDA and SCL), making it extremely efficient for projects where GPIO pins are limited or need to be optimized.

In the Automated Smart Waste Segregation System with Live Monitoring and Intelligent Bin Alerts, the I2C LCD is used to display real-time waste type detection, bin status, and system alerts. It enhances the user interface by providing clear and readable information on the system's operation.

Wiring Connections:

I2C LCD to Arduino Mega:

- **VCC** → 5V
- **GND** → GND
- **SDA** → Pin **20** on Arduino Mega
- **SCL** → Pin **21** on Arduino Mega

5.7 Interfacing Ultrasonic Sensors with ESP8266 for Bin Level Monitoring

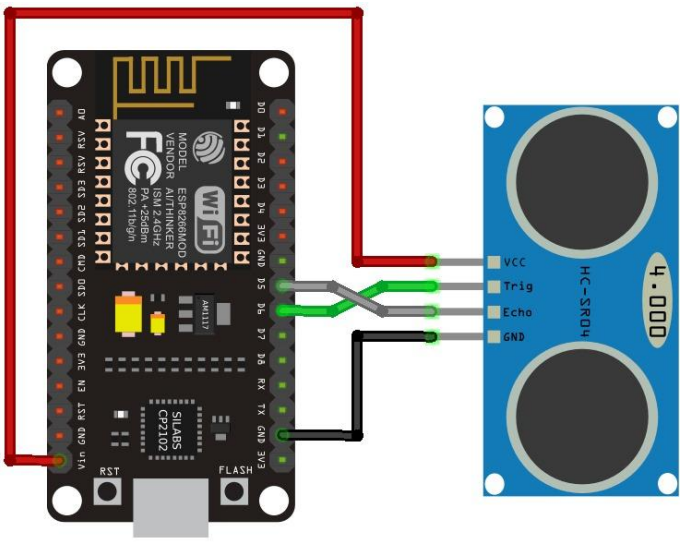


FIG 5.7 Interfacing ultrasonic sensor with ESP8266

The HC-SR04 Ultrasonic Sensor is widely used in embedded systems to measure distance using sound waves. In the Automated Smart Waste Segregation System with Live Monitoring and Intelligent Bin Alerts, ultrasonic sensors are placed inside each bin to monitor the waste fill levels. The ESP8266 (NodeMCU) is used to read the distance data from these sensors and transmit the information wirelessly to a mobile application via Wi-Fi and Firebase.

This enables real-time monitoring of bin levels and helps notify users when a bin is full, thus automating the waste management process and reducing manual supervision.

Wiring Connections:

For each HC-SR04 sensor:

- **VCC** → 5V (external 5V source or level-shifter if needed, as ESP8266 operates at 3.3V logic)
- **GND** → GND (common with ESP8266)
- **Trig** → Digital Pin D6 (GPIO12) *(can be changed as needed)*
- **Echo** → Digital Pin D7 (GPIO13) *(use voltage divider to step down 5V to 3.3V)*

5.8 Integration of ESP8266 NodeMCU and Arduino Mega

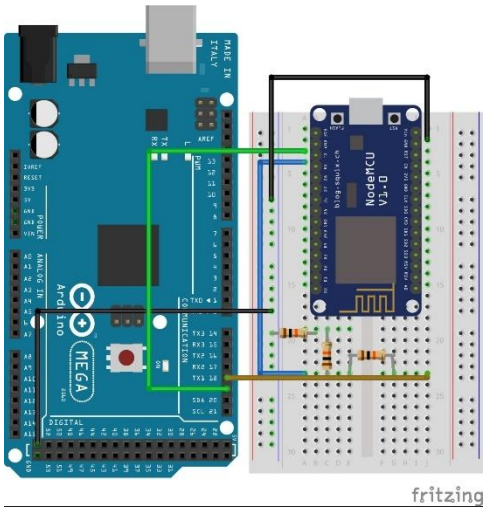


FIG 5.8 Integration of ESP8266 and Arduino Mega

In the Automated Smart Waste Segregation System with Live Monitoring and Intelligent Bin Alerts, both the Arduino Mega and the ESP8266 NodeMCU are used together to divide responsibilities and enhance the overall system performance. The Arduino Mega handles the majority of hardware operations, including controlling motors, stepper drivers, sensors, and actuators. The ESP8266 NodeMCU, on the other hand, is responsible for wireless communication, particularly for sending real-time bin status updates to Firebase and triggering notifications in the mobile application.

Since both devices have their own microcontrollers, the integration between the two is achieved through serial communication (UART). The Arduino Mega sends sensor values (such as ultrasonic bin levels) to the ESP8266, which processes and uploads the data to the cloud.

Wiring Connections (Serial Communication using SoftwareSerial):

Arduino Mega	ESP8266 NodeMCU
TX1 (Pin 18)	RX (D7 / GPIO13)*
RX1 (Pin 19)	TX (D8 / GPIO15)*
GND	GND

CHAPTER 6

RESULTS & VALIDATIONS

6.1 PROTOTYPE OF THE PROPOSED SYSTEM

The prototype is successfully designed and tested. The objectives of the project are satisfactorily realized. Following are the major results obtained. An efficient Automated Smart Waste Segregation System with Live Monitoring and Intelligent Bin Alerts is developed to automatically detect and classify waste using sensors and segregate it into appropriate bins. The system also monitors the bin fill levels in real-time and sends alerts via mobile application using IoT integration.

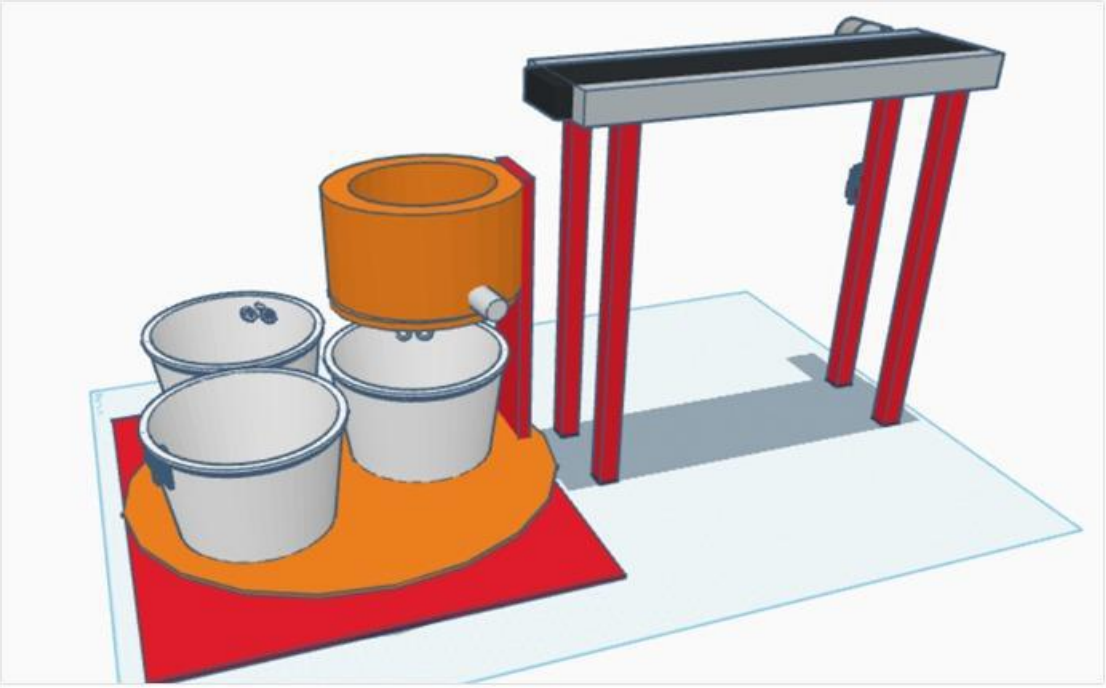


FIG 6.1 Prototype Model

6.2 Waste Detection and Classification Output

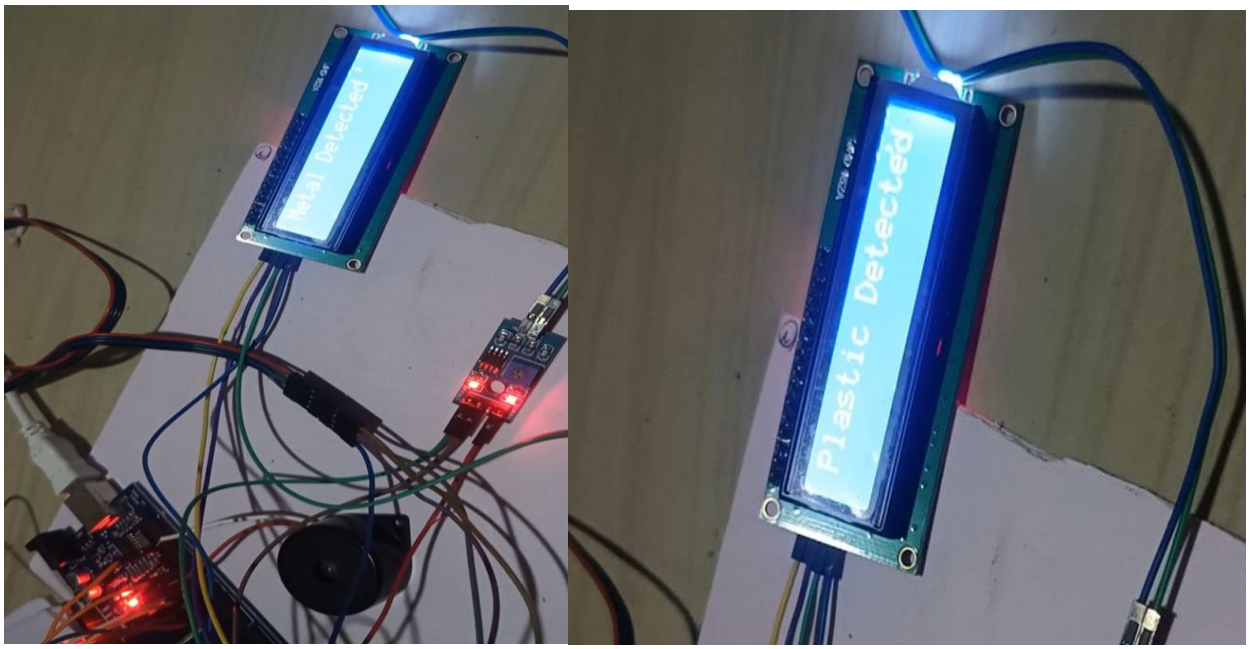


FIG 6.2 Waste detection output

6.3 Firebase Notifications and Mobile App Output

CHAPTER 7

CONCLUSION & FUTURE SCOPE

7.1 CONCLUSION

The Automated Smart Waste Segregation System with Live Monitoring and Intelligent Bin Alerts was successfully developed and implemented using Arduino Mega. The system efficiently identifies and categorizes waste into different types—metallic, wet, and dry—by using appropriate sensors such as inductive, moisture, and ultrasonic sensors. A conveyor mechanism driven by a DC motor ensures proper movement of waste, while a stepper motor rotates to align bins accurately for segregation.

Real-time monitoring is achieved through an I2C LCD that displays the detected waste type, and ultrasonic sensors in each bin help track the fill levels. The ESP8266 module enables wireless communication with a mobile application, which provides live updates and notifications when any bin is about to fill.

Overall, this system minimizes human intervention, enhances waste management efficiency, and contributes to a cleaner and smarter environment through automation and IoT integration.

7.2 FUTURE SCOPE

- **AI-Based Waste Classification:**
  Integrate machine learning models to enhance the accuracy of waste detection and classification beyond basic sensor thresholds.
- **Image Processing Integration:**
  Use camera modules and computer vision techniques to identify different types of waste for improved sorting accuracy.
- **Solar Power Integration:**
  Implement solar panels to power the system, making it energy-efficient and suitable for remote or outdoor areas.
- **Automatic Bin Cleaning Mechanism:**
  Add a self-cleaning system for bins to maintain hygiene and reduce manual cleaning efforts.

- **Mobile App Enhancements:**
  Upgrade the mobile app to include historical data, usage analytics, and customizable notifications for better user control and monitoring.
- **Modular Design for Scalability:**
  Design the system to be modular, allowing easy expansion with additional sensors or bins for industrial-level waste segregation.
- **Cloud Data Storage and Monitoring:**
  Integrate cloud services for real-time monitoring, long-term data storage, and remote management of multiple systems.

REFERENCES

- A. Sharma and P. Kumar, "IoT-Based Smart Waste Segregation and Management System," *International Journal of Engineering Research & Technology (IJERT)*, vol. 9, no. 5, pp. 1120–1124, 2020.
- R. Mehta and S. Verma, "Smart Waste Management System Using IoT," *International Journal of Computer Applications (IJCA)*, vol. 177, no. 28, pp. 1–5, 2020.
- K. Rao and M. Reddy, "Automatic Waste Segregation Using Sensor-Based Technology," *International Journal of Innovative Research in Science, Engineering and Technology (IJIRSET)*, vol. 7, no. 10, pp. 9456–9461, 2018.
- P. Das and R. Sharma, "Design of a Smart Waste Segregator Using Arduino and Sensors," *International Journal of Engineering and Advanced Technology (IJEAT)*, vol. 8, no. 6, pp. 2041–2046, 2019.
- M. Joshi and A. Patel, "Real-Time Waste Monitoring and Management System Using IoT," *International Research Journal of Engineering and Technology (IRJET)*, vol. 6, no. 4, pp. 1204–1208, 2019.
- S. Kumar et al., "IoT-Based Waste Bin Monitoring System for Smart Cities," *International Journal of Scientific & Engineering Research (IJSER)*, vol. 10, no. 2, pp. 456–460, 2019.
- R. Singh and S. Nair, "Implementation of Smart Waste Segregation Using RFID and Sensors," *International Journal of Computer Sciences and Engineering*, vol. 7, no. 3, pp. 228–232, 2019.
- M. Ali and N. Banu, "Design and Development of Smart Waste Sorting System Using Embedded Technology," *International Journal of Advanced Research in Electrical, Electronics and Instrumentation Engineering (IJAREEIE)*, vol. 9, no. 11, pp. 8802–8807, 2020.
- D. Sharma and T. Roy, "A Cloud-Connected Smart Bin for Urban Waste Management," *International Journal of Recent Technology and Engineering (IJRTE)*, vol. 8, no. 6, pp. 1569–1573, 2020.
- L. John and M. Thomas, "Design of an Intelligent Waste Management System Using Smart Sensors and GSM," *International Journal of Engineering and Technology (IJET)*, vol. 7, no. 3, pp. 110–114, 2018.

- N. Raj and K. Jayanthi, "Development of an IoT-Based Smart Waste Collection System," *International Journal of Computer Applications*, vol. 180, no. 45, pp. 22–26, 2018.

APPENDIX CODE

```
//ARDUINO CODE:

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Servo.h>

// Stepper Motor Pins
#define DIR_PIN 4
#define STEP_PIN 3
#define ENABLE_PIN 5

// Moisture Sensor & Buzzer
#define MOISTURE_PIN A2
#define BUZZER_PIN 12
#define EXTRA_BUZZER_PIN 8
const int moistureThreshold = 1000;

// Metal Sensor Pins
#define METAL_PULSE_PIN A0
#define METAL_CAP_PIN A1
const int metalThreshold = 40;

// Stepper Motor Config
const int stepDelay = 500;
const int stepsToMove = 75;

// Servo Motor
#define SERVO_PIN 9
Servo myServo;

// LCD Setup
LiquidCrystal_I2C lcd(0x27, 16, 2);

// Ultrasonic Sensor Pins for Plastic Detection
#define TRIG_PIN 6
#define ECHO_PIN 7
const int plasticThreshold = 15;  // Adjust based on testing

// L298N Motor Driver Pins
#define ENA 10  // PWM Speed Control
#define IN1 22  // Motor Direction
#define IN2 23

void setup() {
   Serial.begin(115200);
   Wire.begin();
   lcd.init();
   lcd.backlight();
   lcd.setCursor(0, 0);
   lcd.print("Monitoring...");

   // Stepper & Sensors Initialization
```

```
    pinMode(DIR_PIN, OUTPUT);
    pinMode(STEP_PIN, OUTPUT);
    pinMode(ENABLE_PIN, OUTPUT);
    pinMode(MOISTURE_PIN, INPUT);
    pinMode(BUZZER_PIN, OUTPUT);
    pinMode(EXTRA_BUZZER_PIN, OUTPUT);
    digitalWrite(BUZZER_PIN, LOW);
    digitalWrite(EXTRA_BUZZER_PIN, LOW);
    myServo.attach(SERVO_PIN);
    myServo.write(0);
    setupMetalSensor();
    pinMode(TRIG_PIN, OUTPUT);
    pinMode(ECHO_PIN, INPUT);
    digitalWrite(ENABLE_PIN, LOW);

    // DC Motor Initialization
    pinMode(ENA, OUTPUT);
    pinMode(IN1, OUTPUT);
    pinMode(IN2, OUTPUT);

    runMotor(100); // Start DC motor at speed 200

    Serial.println("Monitoring sensors...");
}

void loop() {
    int moistureValue = analogRead(MOISTURE_PIN);
    bool metalDetected = detectMetal();
    int distance = getUltrasonicDistance();

    Serial.print("Moisture Level: "); Serial.print(moistureValue);
    Serial.print(" | Metal: "); Serial.print(metalDetected ? "Detected" : "No Metal");
    Serial.print(" | Plastic Distance: "); Serial.println(distance);

    if (metalDetected) {  // Highest Priority
        Serial.println("Metal Detected - Moving 75 steps Counterclockwise...");
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("Metal Detected");
        activateBuzzer();
        stepMotor(stepsToMove, LOW);
        delay(1000);
        rotateServo(1000);
        delay(1000);
        stepMotor(stepsToMove, HIGH);
    }
    else if (distance < plasticThreshold) {  // Second Priority
        Serial.println("Plastic Detected - Stepper remains idle.");
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("Plastic Detected");
        rotateServo(500);
    }
    else if (moistureValue < moistureThreshold) {  // Lowest Priority
        Serial.println("Moisture Detected - Moving 75 steps Clockwise...");
```

```
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("Moisture Detected");
        activateBuzzer();
        stepMotor(stepsToMove, HIGH);
        delay(1000);
        rotateServo(1000);
        delay(1000);
        stepMotor(stepsToMove, LOW);
    }
    else {
        Serial.println("No Object Detected");
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("No Object");
    }

    delay(2000);
}

void stepMotor(int steps, bool direction) {
    digitalWrite(DIR_PIN, direction);
    for (int i = 0; i < steps; i++) {
        digitalWrite(STEP_PIN, HIGH);
        delayMicroseconds(stepDelay);
        digitalWrite(STEP_PIN, LOW);
        delayMicroseconds(stepDelay);
    }
}

int getUltrasonicDistance() {
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);
    long duration = pulseIn(ECHO_PIN, HIGH);
    return duration * 0.034 / 2;
}

void rotateServo(int speedDelay) {
    myServo.write(180);
    delay(speedDelay);
    myServo.write(0);
}

void setupMetalSensor() {
    pinMode(METAL_PULSE_PIN, OUTPUT);
    digitalWrite(METAL_PULSE_PIN, LOW);
    pinMode(METAL_CAP_PIN, INPUT);
}

bool detectMetal() {
    int minval = 1023, maxval = 0;
    long unsigned int sum = 0;
```

```
  for (int imeas = 0; imeas < 128; imeas++) {
    pinMode(METAL_CAP_PIN, OUTPUT);
    digitalWrite(METAL_CAP_PIN, LOW);
    delayMicroseconds(10);
    pinMode(METAL_CAP_PIN, INPUT);
    for (int ipulse = 0; ipulse < 8; ipulse++) {
      digitalWrite(METAL_PULSE_PIN, HIGH);
      delayMicroseconds(2);
      digitalWrite(METAL_PULSE_PIN, LOW);
      delayMicroseconds(2);
    }
    int val = analogRead(METAL_CAP_PIN);
    minval = min(val, minval);
    maxval = max(val, maxval);
    sum += val;
  }
  sum -= minval;
  sum -= maxval;
  static long int sumsum = 0, skip = 0, diff = 0;
  if (sumsum == 0) sumsum = sum << 6;
  long int avgsum = (sumsum + 32) >> 6;
  diff = sum - avgsum;
  if (abs(diff) < (avgsum >> 10)) {
    sumsum = sumsum + sum - avgsum;
    skip = 0;
  } else {
    skip++;
  }
  if (skip > 32) {
    sumsum = sum << 6;
    skip = 0;
  }
  return abs(diff) > metalThreshold;
}

void activateBuzzer() {
  digitalWrite(BUZZER_PIN, HIGH);
  digitalWrite(EXTRA_BUZZER_PIN, HIGH);
  delay(1000);
  digitalWrite(BUZZER_PIN, LOW);
  digitalWrite(EXTRA_BUZZER_PIN, LOW);
}

// Function to Run DC Motor with Speed Control
void runMotor(int speed) {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  analogWrite(ENA, speed);  // Speed range: 0-255
}
```

```
//ESP8266 CODE:

#include <ESP8266WiFi.h>
#include <FirebaseESP8266.h>
#include <ArduinoJson.h>
#include <WiFiClientSecure.h>

// WiFi Credentials
#define WIFI_SSID "realme 11 Pro+ 5G"
#define WIFI_PASSWORD "w2ikvbaz"

// Firebase Credentials
#define FIREBASE_HOST "https://esp8266-1ab22-default-rtdb.firebaseio.com/"
#define FIREBASE_AUTH "1YzQPGA7Ysc4gq0iC01ypWqxd5OBU76ZpM19VQ57"

// FCM Server Key
#define FCM_SERVER_KEY
"BNDe8G4qUNOVw1kyEVoQwov_XAitBsA05arz10uTk1u8dBDXvuWW7vtPxxHIUakl_RoalK6v
1NaO_raQM-iA9VM"
#define FCM_URL "https://fcm.googleapis.com/fcm/send"

// Ultrasonic Sensor Pins
#define TRIG1 D1
#define ECHO1 D2
#define TRIG2 D5
#define ECHO2 D6
#define TRIG3 D7
#define ECHO3 D8

// L298N Motor Control Pins
#define MOTOR_IN1 D3  // GPIO0
#define MOTOR_IN2 D4  // GPIO2
#define MOTOR_ENA D0  // GPIO16 (PWM Speed Control)

FirebaseData firebaseData;
FirebaseAuth auth;
FirebaseConfig config;
WiFiClientSecure client;

void setup() {
  Serial.begin(115200);

  // Ultrasonic sensor pins
  pinMode(TRIG1, OUTPUT); pinMode(ECHO1, INPUT);
  pinMode(TRIG2, OUTPUT); pinMode(ECHO2, INPUT);
  pinMode(TRIG3, OUTPUT); pinMode(ECHO3, INPUT);

  // L298N Motor Driver pins
  pinMode(MOTOR_IN1, OUTPUT);
  pinMode(MOTOR_IN2, OUTPUT);
  pinMode(MOTOR_ENA, OUTPUT);

  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  Serial.print("Connecting to WiFi...");
  while (WiFi.status() != WL_CONNECTED) {
```

```
        delay(500);
        Serial.print(".");
    }
    Serial.println("\nConnected to WiFi!");

    config.host = FIREBASE_HOST;
    config.signer.tokens.legacy_token = FIREBASE_AUTH;

    Firebase.begin(&config, &auth);
    Firebase.reconnectWiFi(true);

    client.setInsecure();  // Allow HTTPS requests

    startMotor(); // Start the motor continuously
}

// Function to get distance from ultrasonic sensor
float getDistance(int trigPin, int echoPin) {
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    long duration = pulseIn(echoPin, HIGH);
    float distance = duration * 0.034 / 2;
    return distance;
}

// Function to send FCM notification
void sendNotification(String title, String message) {
    Serial.println("Sending Notification...");

    WiFiClientSecure client;
    client.setInsecure(); // Bypass SSL verification

    client.connect("fcm.googleapis.com", 443);
    if (!client.connected()) {
        Serial.println("Connection to FCM failed!");
        return;
    }

    StaticJsonDocument<512> jsonData;
    jsonData["to"] = "/topics/bin_alert";  // Topic-based Notification
    jsonData["priority"] = "high";

    JsonObject notification = jsonData.createNestedObject("notification");
    notification["title"] = title;
    notification["body"] = message;

    String requestBody;
    serializeJson(jsonData, requestBody);

    client.println("POST " FCM_URL " HTTP/1.1");
    client.println("Host: fcm.googleapis.com");
```

```
    client.println("Authorization: key=" FCM_SERVER_KEY);
    client.println("Content-Type: application/json");
    client.println("Content-Length: " + String(requestBody.length()));
    client.println();
    client.print(requestBody);

    Serial.println("Notification Sent!");
}

// Function to start the motor continuously
void startMotor() {
    Serial.println("Starting Motor...");
    digitalWrite(MOTOR_IN1, HIGH);
    digitalWrite(MOTOR_IN2, LOW);
    analogWrite(MOTOR_ENA, 255); // Full speed
}

void loop() {
    float bin1 = getDistance(TRIG1, ECHO1);
    float bin2 = getDistance(TRIG2, ECHO2);
    float bin3 = getDistance(TRIG3, ECHO3);

    Serial.print("Bin 1: "); Serial.println(bin1);
    Serial.print("Bin 2: "); Serial.println(bin2);
    Serial.print("Bin 3: "); Serial.println(bin3);

    Firebase.setFloat(firebaseData, "/bins/bin1", bin1);
    Firebase.setFloat(firebaseData, "/bins/bin2", bin2);
    Firebase.setFloat(firebaseData, "/bins/bin3", bin3);

    // Send notifications if bins are full
    if (bin1 <= 10) sendNotification("Bin 1 Full!", "Please empty Bin 1.");
    if (bin2 <= 10) sendNotification("Bin 2 Full!", "Please empty Bin 2.");
    if (bin3 <= 10) sendNotification("Bin 3 Full!", "Please empty Bin 3.");

    delay(1000); // Delay before next reading
}
```